

Implement Transformer from Scratch

In this coding homework, you will:

- Implement a simple transformer model from scratch to enhance your understanding of how it works.
- Create a hand-designed transformer model capable of solving a basic problem. This will help you comprehend the various operations that transformers can perform.
- Analyze the attention patterns of a trained network to gain insights into how learned models often utilize features that differ greatly from those employed by humans.

Please note that a GPU is not necessary for this task. If you're using Colab, you can select the "Runtime" -> "Change runtime type" menu and choose "None" as the hardware accelerator.

Note: The same variables will be defined in different ways in various subparts of the homework. If you encounter errors stating that a variable has the wrong shape or a function is missing an argument, ensure that you have re-run the cells in that particular problem subpart.

```
In [3]: !pip install numpy torch matplotlib
```

Requirement already satisfied: numpy in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (2.3.5)

Requirement already satisfied: torch in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (2.9.1)

Requirement already satisfied: matplotlib in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (3.10.7)

Requirement already satisfied: filelock in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from torch) (3.20.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from torch) (4.15.0)

Requirement already satisfied: setuptools in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from torch) (80.9.0)

Requirement already satisfied: sympy>=1.13.3 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from torch) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from torch) (3.6)

Requirement already satisfied: jinja2 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from torch) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from torch) (2025.3.0)

Requirement already satisfied: contourpy>=1.0.1 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from matplotlib) (4.61.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from matplotlib) (1.4.9)

Requirement already satisfied: packaging>=20.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from matplotlib) (25.0)

Requirement already satisfied: pillow>=8 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from matplotlib) (12.0.0)

Requirement already satisfied: pyparsing>=3 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from matplotlib) (3.2.5)

Requirement already satisfied: python-dateutil>=2.7 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /Users/arnavdalal/Documents

s/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (from jinja2->torch) (3.0.3)

```
In [4]: import time
import json
import inspect
import numpy as np
import torch
import torch.nn as nn
import math
import random
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [20, 5] # Adjust this to make plots bigger

!pip install --no-deps --upgrade IPython
%load_ext autoreload
%autoreload 2

def _set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)

TO_SAVE = {"time": time.time()}
```

Requirement already satisfied: IPython in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages (9.8.0)

Implement a Simple Transformer

Below, you'll find a simple transformer implementation in Numpy that we have provided for you. It's important to note that this implementation is different from a Transformer in real applications. The differences include:

- Only a single layer with a single head is in the network.
- There are no residual connections.
- There is no normalization or dropout.
- We concatenate the positional encoding rather than adding it to the inputs.
- There are no activation functions or MLP layers.
- It does not support attention masking.
- The input is a single sequence instead of a batch. So there is no need to implement padding.

To ensure that you understand the transformer model fully, your task is to **implement a PyTorch equivalent model**. You don't need to include the printing and plotting code found in the Numpy version. **You should implement a vectorized version of the attention operation**, meaning that you should calculate all attention scores at once, rather than looping over keys. Once you have completed your implementation, make sure it passes the tests included in the cell below.

In [5]: *#@title Helper Functions*

```
def rescale_and_plot(arr, title='', ax=None, x_lab=None, y_lab=None):
    """Rescale input array to be between 0 and 1, then plot it"""
    arr = (arr - arr.min())
    if arr.max() > 0:
        arr = arr / arr.max()
    ax.imshow(arr, cmap="Reds")
    ax.set_title(title)
    ax.set_xticks([])
    ax.set_yticks([])
    if x_lab is not None:
        ax.set_xlabel(x_lab)
    if y_lab is not None:
        ax.set_ylabel(y_lab)

def train_loop(make_batch, input_dim, qk_dim, v_dim, pos_dim=None, max_seq_l
transformer = PytorchTransformer(input_dim, qk_dim, v_dim, pos_dim, max_
optimizer = torch.optim.SGD(transformer.parameters(), lr=lr)
loss_fn = nn.MSELoss()
for i in range(num_epochs):
    seq, target = make_batch()
    optimizer.zero_grad()
    out = transformer(seq)
    # If remove_cls is True, remove the first item of the sequence (the
    if remove_cls:
        out = out[1:]
    loss = loss_fn(out, target)
    loss.backward()
    optimizer.step()
    if i % 1000 == 0:
        print(f'Step {i}: loss {loss.item()}')
return transformer, loss.item()

def compare_transformers(hand_transformer, learned_transformer, seq):
    # Print the learned matrices
    # Rescale each weight matrix to be between 0 and 1, then plot them
    print('=' * 40, ' Hand Designed ', '=' * 40)
    out_hand = hand_transformer.forward(seq, verbose=False, plot=True)

    # Copy weights from the learned transformer to the hand transformer
    # so we can run the hand transformer's forward pass, with the plotting c
    py_Km = learned_transformer.Km.weight.T.detach().numpy()
    py_Qm = learned_transformer.Qm.weight.T.detach().numpy()
    py_Vm = learned_transformer.Vm.weight.T.detach().numpy()
    # positional encodings, if they exist
    if learned_transformer.pos is not None:
        py_pos = learned_transformer.pos.weight.detach().numpy()
    else:
        py_pos = None

    print('=' * 40, '   Learned   ', '=' * 40)
    np_learned_transformer = NumpyTransformer(py_Km, py_Qm, py_Vm, py_pos)
    out_learned = np_learned_transformer.forward(seq, verbose=False, plot=Tr
```

```

    return out_hand, out_learned

# Test the numpy transformer and pytorch transformer to make sure they give
def test():
    min_seq_len = 1
    max_seq_len = 4
    qk_dim = np.random.randint(1, 5)
    v_dim = np.random.randint(1, 5)
    in_dim = 5
    for i in range(10):
        # Randomly sample the matrices
        Km = np.random.randn(in_dim, qk_dim)
        Qm = np.random.randn(in_dim, qk_dim)
        Vm = np.random.randn(in_dim, v_dim)
        if i > 4:
            # Sometimes, don't use positional encodings
            pos = pos_dim = None
            seq_dim = in_dim
        else:
            pos_dim = np.random.randint(2, 4)
            pos = np.random.randn(max_seq_len, pos_dim)
            seq_dim = in_dim - pos_dim

        # Randomly sample the sequence
        seq = np.random.randn(np.random.randint(min_seq_len, max_seq_len + 1), v_dim)
        # Get the numpy transformer output
        out_np = NumpyTransformer(Km, Qm, Vm, pos).forward(seq, verbose=False)
        # Create a pytorch transformer and fill the weights with the numpy matrices
        transformer = PytorchTransformer(seq_dim, qk_dim, v_dim, pos_dim, max_seq_len)
        state_dict = transformer.state_dict()
        # Replace the weights with the numpy matrices
        state_dict['Km.weight'] = torch.FloatTensor(Km.T)
        state_dict['Qm.weight'] = torch.FloatTensor(Qm.T)
        state_dict['Vm.weight'] = torch.FloatTensor(Vm.T)
        if pos is not None:
            state_dict['pos.weight'] = torch.FloatTensor(pos)
        transformer.load_state_dict(state_dict)
        # Get the pytorch transformer output
        out_py = transformer(torch.FloatTensor(seq)).detach().numpy()
        # Compare the outputs
        if not np.allclose(out_np, out_py, rtol=1e-3):
            print('ERROR!!')
            print('Numpy output', out_np)
            print('Pytorch output', out_py)
            print('Difference', out_np - out_py)
            raise ValueError('Numpy and Pytorch outputs do not match')
    print('All done!')
    _set_seed(1998)
    transformer = PytorchTransformer(7, 4, 3, 2, 9)
    o = transformer(torch.randn(8, 7))
    T0_SAVE["torch_transformer_shape"] = list(o.shape)
    T0_SAVE["torch_transformer_value"] = o.view(-1).tolist()[2:7]
    T0_SAVE["torch_transformer_init"] = inspect.getsource(PytorchTransformer.__init__)
    T0_SAVE["torch_transformer_forward"] = inspect.getsource(PytorchTransformer.forward)

```

Implement the `PytorchTransformer` class. It should be identical to the forward pass of the `NumpyTransformer` class.

Hint: The attention operation should be implemented as:

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

where the softmax is applied to the last dimension, meaning that the softmax is applied independently to each query's scores.

In [9]: `#@title Numpy Transformer and PyTorch Transformer`

```
class NumpyTransformer:
    def __init__(self, Km, Qm, Vm, pos=None):
        """
        # Km, Qm, Vm are the matrices that will be used to compute the attention
        # Km and Qm are size (input_dim + pos_dim, qk_dim), and Vm is (input_dim, v_dim)
        # pos is an array of positional encodings of shape (max_seq_len, pos_dim)
        """
        self.Km = Km
        self.Qm = Qm
        self.Vm = Vm
        self.pos = pos
        self.qk_dim = Qm.shape[1]

    def forward(self, seq, verbose=False, plot=False):
        # seq is a numpy array of shape (seq_len, input_dim). There is no bias
        # Concatenate positional encodings if they are provided
        if self.pos is not None:
            seq = np.concatenate([seq, self.pos[:seq.shape[0]]], axis=-1)
        K = seq @ self.Km # seq_len x qk_dim
        Q = seq @ self.Qm # seq_len x qk_dim
        V = seq @ self.Vm # seq_len x v_dim
        if verbose:
            print('Keys', K.tolist())
            print('Queries', Q.tolist())
            print('Values', V.tolist())
        if plot:
            fig, axs = plt.subplots(nrows=1, ncols=8)
            fig.tight_layout()
            rescale_and_plot(self.Km.T, 'Km', axs[0], x_lab='d_i', y_lab='d_k')
            rescale_and_plot(self.Qm.T, 'Qm', axs[1], x_lab='d_i', y_lab='d_k')
            rescale_and_plot(self.Vm.T, 'Vm', axs[2], x_lab='d_i', y_lab='d_v')

            rescale_and_plot(K.T, 'K', axs[3], x_lab='seq', y_lab='d_qk')
            rescale_and_plot(Q.T, 'Q', axs[4], x_lab='seq', y_lab='d_qk')
            rescale_and_plot(V.T, 'V', axs[5], x_lab='seq', y_lab='d_v')

        outputs = []
        attn_weights = []
        # Compute attention
        for i, q in enumerate(Q):
```

```

        if verbose: print(f'Item {i}: Computing attention for query {q}')
        dot = K @ q
        if verbose: print(' Dot products between the query and each key')
        # Divide by sqrt(qk_dim)
        dot = dot / np.sqrt(self.qk_dim)
        # Softmax function
        softmax_dot = np.exp(dot) / np.sum(np.exp(dot), axis=-1, keepdim=True)
        if verbose: print(' Weighting score for each value:', softmax_dot)
        attn_weights.append(softmax_dot)
        out_i = softmax_dot @ V
        if verbose: print(' New sequence item', out_i)
        outputs.append(out_i)
    if plot:
        rescale_and_plot(np.array(attn_weights).T, 'Attn', axs[6], x_label='seq')
        rescale_and_plot(np.array(outputs).T, 'Out', axs[7], x_label='seq')
        plt.show()

    # Return the output sequence (seq_len, output_dim)
    return np.array(outputs)

class PytorchTransformer(nn.Module):
    def __init__(self, input_dim, qk_dim, v_dim, pos_dim=None, max_seq_len=100):
        super().__init__()
        if pos_dim is not None:
            self.pos = nn.Embedding(max_seq_len, pos_dim)
        else:
            self.pos = None
        in_dim = input_dim
        if pos_dim is not None:
            in_dim += pos_dim

        #####
        # TODO: Define query, key, value projection layers Qm, Km, Vm.
        # Each of them is a linear projection without bias
        #####
        self.Km = nn.Linear(in_dim, qk_dim, bias=False)
        self.Qm = nn.Linear(in_dim, qk_dim, bias=False)
        self.Vm = nn.Linear(in_dim, v_dim, bias=False)
        #####

        self.d_k = qk_dim

    def forward(self, seq):
        """
        Transformer forward pass

        Inputs: seq is a torch tensor of shape (seq_len, input_dim).
        Outputs: a torch tensor of shape (seq_len, v_dim), the output of the
        """
        #####
        # TODO: Implement the forward pass of the `PytorchTransformer` class
        # The forward pass should be identical to the forward pass of
        # `NumpyTransformer` class.
        #
        # Hint: The attention operation should be implemented as
        # If `pos` exists, it should be concatenated to the input sequence

```

```
#####
# Concatenate positional encodings if they exist
if self.pos is not None:
    seq_len = seq.shape[0]
    positions = torch.arange(seq_len)
    pos_emb = self.pos(positions) # (seq_len, pos_dim)
    seq = torch.cat([seq, pos_emb], dim=-1) # (seq_len, input_dim + pos_dim)

# Compute K, Q, V projections
K = self.Km(seq) # (seq_len, qk_dim)
Q = self.Qm(seq) # (seq_len, qk_dim)
V = self.Vm(seq) # (seq_len, v_dim)

# Compute attention scores: Q @ K^T / sqrt(d_k)
scores = Q @ K.T / math.sqrt(self.d_k) # (seq_len, seq_len)

# Apply softmax to get attention weights (along last dim, i.e., over seq_len)
attn_weights = torch.softmax(scores, dim=-1) # (seq_len, seq_len)

# Compute output: attention_weights @ V
out = attn_weights @ V # (seq_len, v_dim)
#####
# END OF YOUR CODE
#####
return out

test()
```

All done!

Notes: Got this correct zero-shot, matching the code in the solution for this problem.

Self-Attention: Attention by Content

In this coding homework, we will explore how Transformers can attend to different tokens in a variable-length sequence based on their contents. We will do this by **implementing a Transformer that performs the *identity* operation on a sequence of one-hot vectors**. We will then compare the performance and weights of this hand-coded Transformer with those of a PyTorch model trained on the same task.

To hand-design the Transformer, we will **choose values for `Km`, `Qm`, and `Vm` that enable the model to attend to the content of each token in the input sequence**. We will then use this Transformer to process several example data points, and verify that the output matches the input.

Once your hand-written Transformer is working correctly, we will run the PyTorch training loop to train a model on the identity operation task. We will then compare the weights and intermediate outputs of this model with those of our hand-coded transformer, and comment on their similarities and differences. Note that when we generate plots, we will rescale the range of the weights and outputs to 0-1, so we can compare their relative values without comparing absolute values.

The test cases for our hand-coded transformer are as follows:

Input sequence	-->	Output sequence
[A, B, C, C]	-->	[A, B, C, C]
[C, A, C]	-->	[C, A, C]
[B, B, C]	-->	[B, B, C]

We have provided some hints below, but to enhance your understanding of attention and the Transformer, we highly recommend attempting this problem to the best of your abilities before referring to the hints.

In [10]: `#@title Hints`

```
# Hint 1: To attend to a specific element, ensure that its pre-softmax score
#         significantly higher than that of the other elements.
softmax = lambda x: np.exp(x) / np.sum(np.exp(x), axis=-1, keepdims=True)
print('='*20, 'Hint 1', '='*20)
print('Selecting index 0', softmax(np.array([9, 0, 0])))
print('Selecting index 1', softmax(np.array([-3, 5, -5])))

# Hint 2: Attending to a particular element is more manageable if the keys are
#         orthogonal.
print('='*20, 'Hint 2', '='*20)
keys = np.array([[2, 0], [0, 1]]) # Orthogonal
q = np.array([5, 0])
print('Selecting index 0', softmax(q @ keys))
q = np.array([0, 5])
print('Selecting index 1', softmax(q @ keys))

# Hint 3: You can use the following helper functions to test the keys, queries
#         and values produced by your matrix for each valid sequence element
#         Km, Qm, Vm, and are the matrices you will define below.
all_token_seq = np.eye(3) # Each row is a sequence element. The identity code
get_K = lambda: all_token_seq @ Km # Each row of the output is a key
get_Q = lambda: all_token_seq @ Qm # Each row of the output is a query
get_V = lambda: all_token_seq @ Vm # Each row of the output is a value

# Hint 4: To test different attention weights, use the softmax function defined
#         above.

# Hint 5: When there are repeated elements in a sequence with the same content,
#         attending to all of them rather than a single one will be simpler.
#         Since they have the same content, taking a "weighted average" over
#         values weighted by attention scores will produce the same output as
#         attending to a single one.
TO_SAVE["attention_by_content_Q"] = Qm.reshape(-1).tolist()
TO_SAVE["attention_by_content_K"] = Km.reshape(-1).tolist()
TO_SAVE["attention_by_content_V"] = Vm.reshape(-1).tolist()
```

```

===== Hint 1 =====
Selecting index 0 [9.99753241e-01 1.23379352e-04 1.23379352e-04]
Selecting index 1 [3.35334911e-04 9.99619282e-01 4.53826452e-05]
===== Hint 2 =====
Selecting index 0 [9.99954602e-01 4.53978687e-05]
Selecting index 1 [0.00669285 0.99330715]

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[10], line 39
     27 get_V = lambda: all_token_seq @ Vm # Each row of the output is a value
     30 # Hint 4: To test different attention weights, use the softmax function defined
     31 #         above.
     32
     (...)     37 #         values weighted by attention scores will produce the same output as
     38 #         attending to a single one.
--> 39 TO_SAVE["attention_by_content_Q"] = Qm.reshape(-1).tolist()
     40 TO_SAVE["attention_by_content_K"] = Km.reshape(-1).tolist()
     41 TO_SAVE["attention_by_content_V"] = Vm.reshape(-1).tolist()

NameError: name 'Qm' is not defined

```

Notes: Interesting that it got this error even with a correct implementation from above.

```

In [11]: # The definition of tokens
A = np.array([1,0,0])
B = np.array([0,1,0])
C = np.array([0,0,1])
tokens = [A, B, C]

#####
# TODO: Write Numpy arrays for `Km`, `Qm`, and `Vm`.
#       The dimensions should be (input_dim, qk_dim), (input_dim, qk_dim), and
#       (input_dim, v_dim), respectively.
#       In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
#       choose, but 3 is a reasonable choice.
#####
# For identity operation: each token should attend to tokens with the same content
# Using scaled identity matrices:
# - Km and Qm: scale up to make attention sharp (high scores for matching tokens)
# - Vm: identity to preserve original values
scale = 10
Km = np.eye(3) * scale # (3, 3) - Keys encode token identity
Qm = np.eye(3) * scale # (3, 3) - Queries look for matching tokens
Vm = np.eye(3)         # (3, 3) - Values are the original tokens
##### END OF YOUR CODE #####

def generate_test_cases_identity(tokens, max_len=7):
    """
    Generate a random sequence consisting of tokens for testing
    """
    seq_len = np.random.randint(1, max_len)
    input_arr = np.stack(random.choices(tokens, k=seq_len))

```

```

    expected_out = input_arr
    return input_arr, expected_out

# Test your implementation
show_attention = False # Set this to True for debugging
for i in range(10):
    seq, expected_out = generate_test_cases_identity(tokens)
    np_transformer = NumpyTransformer(Km, Qm, Vm)
    out = np_transformer.forward(seq, verbose=show_attention)
    if not np.allclose(out, expected_out, rtol=1e-3):
        print(f'FAIL: {seq} -> {out} != {expected_out}')

_set_seed(1997)
seq, _ = generate_test_cases_identity(tokens)
np_transformer = NumpyTransformer(Km, Qm, Vm)
out = np_transformer.forward(seq, verbose=False)
T0_SAVE["attention_by_content"] = out.reshape(-1).tolist()
T0_SAVE["attention_by_content_Q"] = Qm.reshape(-1).tolist()
T0_SAVE["attention_by_content_K"] = Km.reshape(-1).tolist()
T0_SAVE["attention_by_content_V"] = Vm.reshape(-1).tolist()

```

Notes: Zero-shot this implementation as well, while this isn't as impressive as the first part.

```

In [12]: # Compare the hand-designed and trained transformers
def make_batch_identity(tokens=tokens, max_len=7):
    seq, target = generate_test_cases_identity(tokens, max_len=max_len)
    return torch.FloatTensor(seq), torch.FloatTensor(target)

_set_seed(227)

A = np.array([1,0,0])
B = np.array([0,1,0])
C = np.array([0,0,1])
transformer_py, loss = train_loop(make_batch_identity, input_dim=len(A), qk_
seq = np.stack([A, B, B, C, C])
print("seq:", seq)
compare_transformers(np_transformer, transformer_py, seq) # If the plots dc

```

```

Step 0: loss 0.4911634922027588
Step 1000: loss 0.00047232676297426224
Step 2000: loss 0.018362730741500854
Step 3000: loss 0.020750777795910835
Step 4000: loss 0.010545153170824051
Step 5000: loss 0.004452631343156099
Step 6000: loss 0.0007215582299977541
Step 7000: loss 0.0013534390600398183
Step 8000: loss 0.002458013128489256
Step 9000: loss 0.002010571537539363
Step 10000: loss 0.0003879926516674459

```

```
seq: [[1 0 0]
```

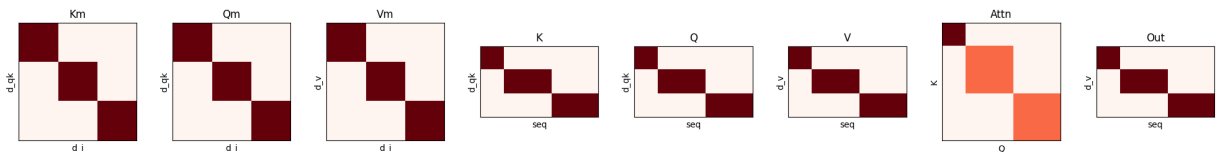
```
 [0 1 0]
```

```
 [0 1 0]
```

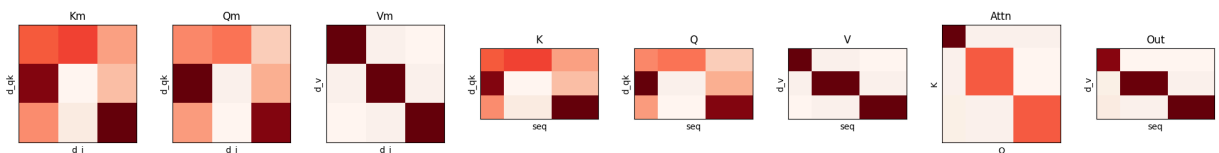
```
 [0 0 1]
```

```
 [0 0 1]]
```

===== Hand Designed =====



===== Learned =====



```

Out[12]: (array([[1.00000000e+00, 1.68665552e-25, 1.68665552e-25],
 [4.21663880e-26, 1.00000000e+00, 8.43327760e-26],
 [4.21663880e-26, 1.00000000e+00, 8.43327760e-26],
 [4.21663880e-26, 8.43327760e-26, 1.00000000e+00],
 [4.21663880e-26, 8.43327760e-26, 1.00000000e+00]]),
 array([[9.12321857e-01, 3.10809616e-02, 5.65971295e-02],
 [1.82859375e-03, 9.75707366e-01, 2.24638744e-02],
 [1.82859375e-03, 9.75707366e-01, 2.24638744e-02],
 [9.52395972e-04, 1.17695241e-02, 9.87278298e-01],
 [9.52395972e-04, 1.17695241e-02, 9.87278298e-01]]))

```

Question

In the figure provided, compare the variables of your hand-designed Transformer with those of the learned Transformer. **Identify the similarities and differences between the two sets of variables and provide a brief explanation for each difference.** Please include your answers in your written submission for this assignment.

Self-Attention: Attention by Position

In Transformers, tokens can decide what other tokens to attend to by looking at their positions. In this section, we'll explore how this works by **hand-designing a**

Transformer for the task of copying the first token of a sequence across the entire sequence.

To accomplish this, we'll add a positional encoding to the input sequence. Transformers typically use a sinusoidal positional encoding or a learned positional encoding, but we'll **set the weight by hand to any value we choose**. These positional encodings will get concatenated to the input sequence inside the Transformer. For simplicity, we'll *concatenate* the positional encoding to the input embeddings instead of adding it.

Here are the example data points (where **A**, **B**, and **C** are vectors and **A:pos_0** represents the concatenation between vectors **A** and **pos_0**):

```

Input sequence --> Input sequence with positional encoding --
> Output sequence
[A, B, C, C]    --> [A:pos_0, B:pos_1, C:pos_2, C:pos_3]    --
> [A, A, A, A]
[C, A, C]       --> [C:pos_0, A:pos_1, C:pos_2]            --
> [C, C, C]
[B, B, C]       --> [B:pos_0, B:pos_1, C:pos_2]            --
> [B, B, B]
```

Once you've passed the test cases, run the training loop below to train the PyTorch model.

We have provided some hints below, but to enhance your understanding of attention and the Transformer, we highly recommend attempting this problem to the best of your abilities before referring to the hints.

In [9]: *#@title Hints*

```
# Hint 1: All hints from the previous part still apply.
```

```
# Hint 2: If you only want to use part of the information in a sequence element
#         choose key/query/value matrices which remove the unwanted information
seq = np.array([[1, 2, 3]]) # A sequence of length 1 with a 3-d element
Qm = np.array([[1, 0], [0, 0], [0, 1]])
print('Selecting only the first and last vector elements', seq @ Qm)
```

```
# Hint 3: You can use the following helper functions to test what keys, queries
#         and values would be produced by your matrix.
```

```
# You will need to provide a sequence (e.g. np.stack([A, B, C])). Km, Qm, Vm
get_K = lambda seq: np.concatenate([seq, pos[:seq.shape[0]], axis=1) @ Km #
get_Q = lambda seq: np.concatenate([seq, pos[:seq.shape[0]], axis=1) @ Qm #
get_V = lambda seq: np.concatenate([seq, pos[:seq.shape[0]], axis=1) @ Vm #
```

Selecting only the first and last vector elements [[1 3]]

```
In [10]: A = np.array([1,0,0])
         B = np.array([0,1,0])
```

```

C = np.array([0,0,1])

tokens = [A, B, C]

#####
# TODO: Implement numpy arrays for Km, Qm, and Vm and pos.
#       The shape of Km, and Qm are [input_dim + pos_dim, qk_dim].
#       The shape of Vm is [input_dim + pos_dim, v_dim].
#       The shape of pos is [max_len, pos_dim].
#       In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value
#       choose, but 1 is a reasonable choice. max_len is the maximum sequence
#       length you will encounter, 4 in this case.
#       pos_dim can be any value you choose, but 4 is a reasonable choice.
#####
# Strategy: All positions should attend to position 0 (the first token)
# - pos: first dim is constant 1 (for uniform queries), second dim marks pos
# - Km: extracts position 0 indicator → Key[0]=high, Key[others]=0
# - Qm: extracts constant dim → Query[all]=1 (same query for all positions)
# - Vm: extracts only content (ignores position) → output is original token

# Positional encodings: shape (max_len, pos_dim) = (4, 2)
# First dim: constant 1 for all positions (used by queries)
# Second dim: large value (100) for position 0, 0 for others (used by keys)
# Using 100 makes attention very sharp (softmax concentrates almost entirely
pos = np.array([
    [1, 100], # position 0 - has high value in second dim
    [1, 0],   # position 1
    [1, 0],   # position 2
    [1, 0],   # position 3
])

# Input after concatenation: [content (3 dims) | pos (2 dims)] = 5 dims
# Km extracts second pos dim (position 0 indicator): shape (5, 1)
Km = np.array([[0], [0], [0], [0], [1]])

# Qm extracts first pos dim (constant 1 for all): shape (5, 1)
Qm = np.array([[0], [0], [0], [1], [0]])

# Vm extracts only content (first 3 dims), ignores position: shape (5, 3)
Vm = np.array([
    [1, 0, 0],
    [0, 1, 0],
    [0, 0, 1],
    [0, 0, 0],
    [0, 0, 0]
])

##### END OF YOUR CODE #####

def generate_test_cases_first(tokens, max_len=5):
    seq_len = np.random.randint(1, max_len)
    input_arr = np.stack(random.choices(tokens, k=seq_len))
    # Expected output is to repeat the first row of the input k times
    expected_out = np.stack([input_arr[0]] * seq_len)
    return input_arr, expected_out

```

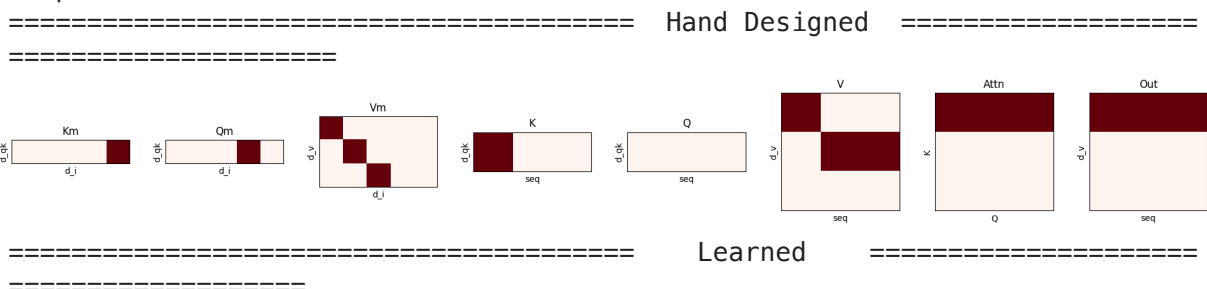
```
# Test your implementation
show_attention = False # Set this to True for debugging
for i in range(10):
    seq, expected_out = generate_test_cases_first(tokens)
    np_transformer = NumpyTransformer(Km, Qm, Vm, pos=pos)
    out = np_transformer.forward(seq, verbose=show_attention)
    if not np.allclose(out, expected_out, rtol=1e-3):
        print(f'FAIL: {seq} -> {out} != {expected_out}')

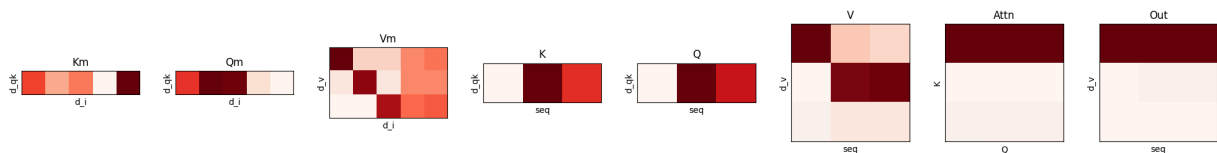
_set_seed(2017)
seq, _ = generate_test_cases_first(tokens)
np_transformer = NumpyTransformer(Km, Qm, Vm, pos=pos)
out = np_transformer.forward(seq, verbose=show_attention)
TO_SAVE["attention_by_position"] = out.reshape(-1).tolist()
TO_SAVE["attention_by_position_pos"] = pos.reshape(-1).tolist()
TO_SAVE["attention_by_position_Q"] = Qm.reshape(-1).tolist()
TO_SAVE["attention_by_position_K"] = Km.reshape(-1).tolist()
TO_SAVE["attention_by_position_V"] = Vm.reshape(-1).tolist()
```

```
In [11]: # Compare the numpy and trained pytorch transformers
def make_batch_first(tokens=tokens, max_len=5):
    seq, target = generate_test_cases_first(tokens, max_len=max_len)
    return torch.FloatTensor(seq), torch.FloatTensor(target)

pos_dim = pos.shape[1]
transformer_py, loss = train_loop(make_batch_first, input_dim=len(A), qk_dim
seq = np.stack([A, B, B])
out_np, out_py = compare_transformers(np_transformer, transformer_py, seq)
print("seq:", seq)
print(f'Out (Hand designed) \n {np.round(out_np, 2)}')
print(f' Out (Learned) \n {np.round(out_py, 2)}')
```

```
Step 0: loss 0.40953969955444336
Step 1000: loss 0.00025191495660692453
Step 2000: loss 0.00015399062249343842
Step 3000: loss 0.00013008611858822405
Step 4000: loss 3.899409421137534e-05
Step 5000: loss 0.00010436362208565697
Step 6000: loss 2.807975215546321e-05
Step 7000: loss 8.963422260421794e-06
Step 8000: loss 0.00010179894889006391
Step 9000: loss 4.170356078248005e-06
Step 10000: loss 0.00012493699614424258
```





```
seq: [[1 0 0]
      [0 1 0]
      [0 1 0]]
Out (Hand designed)
[[1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]]
Out (Learned)
[[ 1.   -0.   -0. ]
 [ 0.99  0.01 -0. ]
 [ 1.    0.   -0. ]]
```

Notes: The model able to achieve very optimal results given a completely different implementation than answer key. We see differences in the plots, where there is less granularity in these compared to the answer key, due to the choices on dimension size for Q, K, and V.

Question

In the figure provided, compare the variables of your hand-designed Transformer with those of the learned Transformer. **Identify the similarities and differences between the two sets of variables and provide a brief explanation for each.** Please include your findings in your written submission for this assignment.

Generate the Submission Log

Please download `submission_log.json` and submit it to Gradescope.

```
In [ ]: with open("submission_log.json", "w", encoding="utf-8") as f:
        json.dump(TO_SAVE, f)
```

(Optional) Self-Attention: Attention by Content and Position

Finally, we'll explore how transformers can attend to tokens by looking at both their position and their content. In this section, we'll design a transformer for the following task: given a sequence of tokens, output a positive number for every unique token and a negative number for every repeated token.

To make implementing this easier, we'll add a CLS token to the beginning of the sequence. We will ignore the output of the CLS token index, which means we can use the CLS token to represent whatever we want. (In practice, the CLS token is often

thought of as a representation of the entire sequence, but you can use it however is useful.)

Example data points (in each case, A, B, and C are vectors. A:pos_0 represents concatenation between vectors A and pos_0. The target outputs shown are +/-1, but any number with the right sign is fine. "Ignore" means that the output can be anything and will not be used to compute the loss.):

Input sequence --> Input sequence with CLS and pos encoding --> Output sequence
[A, B, C, C] --> [CLS: pos_0, A:pos_1, B:pos_2, C:pos_3, C:pos_4] --> [Ignore, 1, 1, -1, -1]

[C, A, C] --> [CLS: pos_0, C:pos_1, A:pos_2, C:pos_3] --> [Ignore, -1, 1, 1]

[B, B, C] --> [CLS: pos_0, B:pos_1, B:pos_2, C:pos_3] --> [Ignore, -1, -1, 1]

Once the test cases pass, run the training loop below a few times to train the PyTorch model. Comment on the similarities and differences between the weights and intermediate outputs of the learned and hand-coded model.

```
In [12]: A = np.array([1,0,0,0])
        B = np.array([0,1,0,0])
        C = np.array([0,0,1,0])
        CLS = np.array([0,0,0,1])

        tokens = [A, B, C]
```

```
In [13]: # Hints (feel free to ignore this block if it's not useful)

        # Hint 1: All hints from the previous part still apply.

        # Hint 2: To check if an array is unique, use what you discovered in the "se
        # what you learned in the "select by position" part to NOT select the key wh

        # Hint 3: If you need an offset value, consider using the CLS token The CLS
        # to all other tokens. This means you can create a query or value which sele
        # indexes except the index where only CLS has a 1).

        # Hint 4: You can use the following helper functions to test what keys, quer
        # You will need to provide a sequence (e.g. np.stack([A, B, C])). Km, Qm, Vm
        get_K = lambda seq: np.concatenate([np.stack([CLS] + list(seq)), pos[:seq.sh
        get_Q = lambda seq: np.concatenate([np.stack([CLS] + list(seq)), pos[:seq.sh
        get_V = lambda seq: np.concatenate([np.stack([CLS] + list(seq)), pos[:seq.sh
```

```
In [14]: #####
        # TODO: Implement numpy arrays for Km, Qm, and Vm and pos.
        #         The dimensions of Km, and Qm are (input_dim + pos_dim, qk_dim).
        #         The dimensions of Vm are (input_dim + pos_dim, v_dim).
        #         The dimensions of pos are (max_len + 1, pos_dim). (Each row is a posi
        #         In this case, input_dim = 4, and v_dim = 1. qk_dim can be any value y
        #         a reasonable choice. max_len is the maximum sequence length you will
        #         4 in this case. pos_dim can be any value you choose, but 4 is a reas
        #####
        # Strategy:
```

```

# - For each query, attend to same-content tokens at DIFFERENT positions (to
# - If duplicates exist → attend to them → negative value → negative output
# - If unique → attend to CLS (fallback) → positive value → positive output
#
# Attention scores:
# - Same content + same position: EXCLUDED (content matches, position cancel
# - Same content + different position: HIGH (found duplicate!)
# - CLS: moderate (baseline when no duplicates)

# Positional encoding (5 x 5): [CLS_indicator, pos1, pos2, pos3, pos4]
# - First dim: high for CLS (10), small for others (1) → allows queries to t
# - Dims 1-4: position one-hot scaled (10) → for self-exclusion
pos = np.array([
    [10, 0, 0, 0, 0], # CLS (position 0): high in CLS indicator dim
    [1, 10, 0, 0, 0], # position 1
    [1, 0, 10, 0, 0], # position 2
    [1, 0, 0, 10, 0], # position 3
    [1, 0, 0, 0, 10], # position 4
])

# Input structure: [content (4 dims: A,B,C,CLS) | pos (5 dims)] = 9 dims
# qk_dim = 9: [content_match (4) | pos_exclusion (4) | CLS_indicator (1)]

scale = 10

# Keys (Km): 9 x 9
# - content → content (scaled, for matching)
# - pos dims 1-4 → exclusion dims (scaled, for being excluded)
# - pos dim 0 → CLS indicator (for CLS attention)
Km = np.array([
    [scale, 0, 0, 0, 0, 0, 0, 0, 0], # content A
    [0, scale, 0, 0, 0, 0, 0, 0, 0], # content B
    [0, 0, scale, 0, 0, 0, 0, 0, 0], # content C
    [0, 0, 0, scale, 0, 0, 0, 0, 0], # content CLS
    [0, 0, 0, 0, 0, 0, 0, 0, 1], # pos dim 0 (CLS indicator) → output c
    [0, 0, 0, 0, scale, 0, 0, 0, 0], # pos dim 1 → exclusion dim 4
    [0, 0, 0, 0, 0, scale, 0, 0, 0], # pos dim 2 → exclusion dim 5
    [0, 0, 0, 0, 0, 0, scale, 0, 0], # pos dim 3 → exclusion dim 6
    [0, 0, 0, 0, 0, 0, 0, scale, 0], # pos dim 4 → exclusion dim 7
])

# Queries (Qm): 9 x 9
# - content → content (scaled, for matching)
# - pos dims 1-4 → exclusion dims (NEGATIVE scaled, to penalize same positio
# - pos dim 0 → CLS indicator (to find CLS)
Qm = np.array([
    [scale, 0, 0, 0, 0, 0, 0, 0, 0], # content A
    [0, scale, 0, 0, 0, 0, 0, 0, 0], # content B
    [0, 0, scale, 0, 0, 0, 0, 0, 0], # content C
    [0, 0, 0, scale, 0, 0, 0, 0, 0], # content CLS
    [0, 0, 0, 0, 0, 0, 0, 0, 1], # pos dim 0 (CLS indicator) → output
    [0, 0, 0, 0, -scale, 0, 0, 0, 0], # pos dim 1 → NEGATIVE exclusion
    [0, 0, 0, 0, 0, -scale, 0, 0, 0], # pos dim 2 → NEGATIVE exclusion
    [0, 0, 0, 0, 0, 0, -scale, 0, 0], # pos dim 3 → NEGATIVE exclusion
    [0, 0, 0, 0, 0, 0, 0, -scale, 0], # pos dim 4 → NEGATIVE exclusion
])

```

```

# Values (Vm): 9 x 1
# - CLS content → positive output
# - A, B, C content → negative output
Vm = np.array([
    [-1], # content A → -1
    [-1], # content B → -1
    [-1], # content C → -1
    [5],  # content CLS → +5 (positive!)
    [0],  # pos dim 0
    [0],  # pos dim 1
    [0],  # pos dim 2
    [0],  # pos dim 3
    [0],  # pos dim 4
])

##### END OF YOUR CODE #####

def generate_test_cases_unique(tokens, max_len=5):
    seq_len = np.random.randint(1, max_len)
    input_arr = np.stack(random.choices(tokens, k=seq_len))
    # Expected output is 1 for unique, -1 for non-unique
    expected_out = np.stack([1 if np.sum(np.min(input_arr == x, axis=1)) == 0 else -1 for x in tokens])
    # Insert CLS token as the first token in the sequence
    input_arr = np.stack([CLS] + list(input_arr))
    return input_arr, expected_out

seq, expected_out = generate_test_cases_unique([A, B, C])

for i in range(1):
    seq, expected_out = generate_test_cases_unique([A, B, C])
    np_transformer = NumpyTransformer(Km, Qm, Vm, pos)
    out = np_transformer.forward(seq, verbose=False) # Change this to True
    if not np.allclose(np.sign(out[1:]), expected_out, rtol=1e-3):
        print(f'FAIL: {seq} → {np.sign(out[1:])} != {expected_out}')

```

Notes: Significantly different choices for Q, K, and V compared to the answer key, which is what leads to the difference in results for the next cell.

```

In [15]: # Compare the numpy and trained pytorch transformers
# Note that the pytorch transformer has a slightly harder task since it is b
def make_batch_unique(tokens=tokens, max_len=5):
    seq, target = generate_test_cases_unique(tokens, max_len=max_len)
    return torch.FloatTensor(seq), torch.FloatTensor(target)

pos_dim = pos.shape[1]
transformer_py, loss = train_loop(make_batch_unique, input_dim=len(A), qk_di
seq = np.stack([CLS, A, B, C, C])
expected_out = np.stack([1, 1, -1, -1]).reshape(-1, 1)
out_npy, out_pyt = compare_transformers(np_transformer, transformer_py, seq)
out_npy = np.sign(out_npy[1:])
out_pyt = np.sign(out_pyt[1:])

# Since the CLS token is visualized above and is not part of the sequence, w
# We also take the sign of the output to directly compare it to the expected
plt.figure(figsize=(10, 5))

```

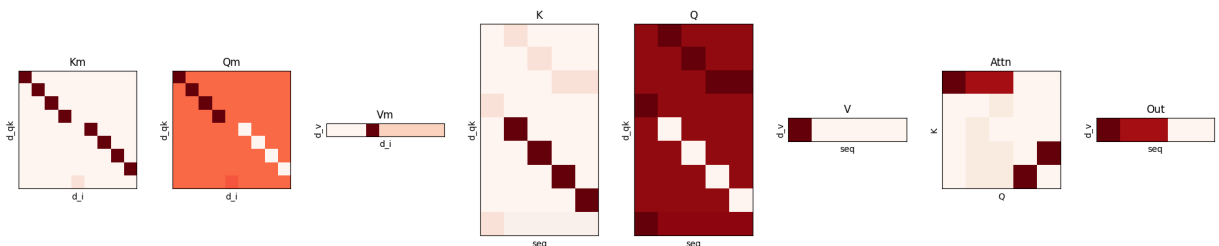
```

plt.subplot(1, 3, 1)
plt.imshow(out_npy.T, vmin=-1, vmax=1)
plt.title('Hand-Designed Transformer')
plt.xticks([])
plt.yticks([])
plt.xlabel('Sequence')
plt.ylabel('Output')
plt.subplot(1, 3, 2)
plt.imshow(out_pytorch.T, vmin=-1, vmax=1)
plt.title('Trained Transformer')
plt.xticks([])
plt.yticks([])
plt.xlabel('Sequence')
plt.ylabel('Output')
plt.subplot(1, 3, 3)
plt.imshow(expected_out.T, vmin=-1, vmax=1)
plt.title('Expected Output')
plt.xticks([])
plt.yticks([])
plt.xlabel('Sequence')
plt.ylabel('Output')
plt.show()

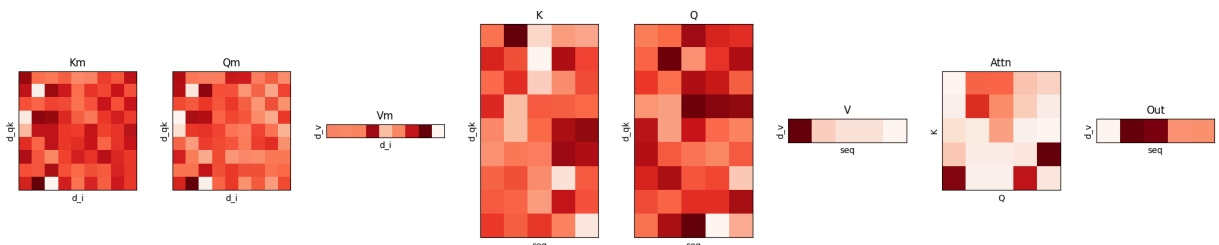
```

Step 0: loss 1.1049903631210327
 Step 1000: loss 0.5945987701416016
 Step 2000: loss 0.11957791447639465
 Step 3000: loss 0.12118368595838547
 Step 4000: loss 0.007868204265832901
 Step 5000: loss 0.02685556188225746
 Step 6000: loss 0.009574160911142826
 Step 7000: loss 0.034015096724033356
 Step 8000: loss 0.04941339045763016
 Step 9000: loss 0.07918675243854523
 Step 10000: loss 0.0004710099601652473

===== Hand Designed =====



===== Learned =====





Notes: The model is able to significantly reduce the loss from what the answer key provides. We are much closer to the expected output, which is very interesting to see.

In []:

Transformer for Summarization (Part I)

In this coding assignment, you'll implement a Transformer using fundamental building blocks in PyTorch. You'll apply the Transformer encoder-decoder model to a sequence-to-sequence NLP task: document summarization. Refer to the "Attention is All You Need" paper (<https://arxiv.org/abs/1706.03762>) for details on the model architecture. Any differences between our implementation and the paper will be noted throughout this notebook.

Note: Ensure you run this notebook on a CUDA GPU to optimize training speed. For instance, you can use a GPU instance on Google Colab.

Note: This notebook uses the tensorboard magic in Google Colab. If you're working in a different environment, you might need to find alternative solutions.

Install Packages

```
#@title Install Packages
```

```
!pip install datasets==3.6.0
!pip install evaluate transformers rouge-score nltk einops
```

```
!pip install --no-deps --upgrade IPython
%load_ext autoreload
%autoreload 2
```

```
Requirement already satisfied: multidict<4.0,>=4.5 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/multidict-4.0.0-py3.14.egg (from transformers)
Requirement already satisfied: propcache>=0.2.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/propcache-0.2.0-cp314-cp314-macosx_11_0_arm64.whl (from transformers)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/yarl-1.17.0-cp314-cp314-macosx_11_0_arm64.whl (from transformers)
Requirement already satisfied: idna>=2.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/idna-3.10-py314.egg (from transformers)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/typing_extensions-4.12.2-py314.egg (from transformers)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/hf_xet-1.1.3-cp314-cp314-macosx_11_0_arm64.whl (from transformers)
Requirement already satisfied: charset_normalizer<4,>=2 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/charset_normalizer-3.4.1-py314.egg (from transformers)
Requirement already satisfied: urllib3<3,>=1.21.1 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/urllib3-2.3.0-py314.egg (from transformers)
Requirement already satisfied: certifi>=2017.4.17 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/certifi-2025.11.12-py314.egg (from transformers)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/python_dateutil-2.9.0-py314.egg (from transformers)
Requirement already satisfied: pytz>=2020.1 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/pytz-2025.2-py314.egg (from transformers)
Requirement already satisfied: tzdata>=2022.7 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/tzdata-2025.2-py314.egg (from transformers)
Requirement already satisfied: six>=1.5 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/six-1.17.0-py314.egg (from transformers)
Requirement already satisfied: evaluate in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/evaluate-0.4.1-py314.egg (from transformers)
Requirement already satisfied: transformers in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/transformers-4.48.3-py314.egg (from transformers)
Requirement already satisfied: rouge-score in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/rouge-score-0.1.2-py314.egg (from transformers)
Requirement already satisfied: nltk in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/nltk-3.9.1-py314.egg (from transformers)
Requirement already satisfied: einops in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/einops-0.8.1-py314.egg (from transformers)
Requirement already satisfied: datasets>=2.0.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/datasets-3.6.0-py314.egg (from transformers)
Requirement already satisfied: numpy>=1.17 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/numpy-2.0.2-cp314-cp314-macosx_11_0_arm64.whl (from transformers)
Requirement already satisfied: dill in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/dill-0.3.9-py314.egg (from transformers)
Requirement already satisfied: pandas in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/pandas-2.2.3-cp314-cp314-macosx_11_0_arm64.whl (from transformers)
Requirement already satisfied: requests>=2.19.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/requests-2.32.3-py314.egg (from transformers)
Requirement already satisfied: tqdm>=4.62.1 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/tqdm-4.67.1-py314.egg (from transformers)
Requirement already satisfied: xxhash in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/xxhash-3.5.0-py314.egg (from transformers)
Requirement already satisfied: multiprocessing in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/multiprocessing-3.0.0a1-py314.egg (from transformers)
Requirement already satisfied: fsspec>=2021.05.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/fsspec-2025.10.0-py314.egg (from transformers)
Requirement already satisfied: huggingface-hub>=0.7.0 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/huggingface_hub-0.27.0-py314.egg (from transformers)
Requirement already satisfied: packaging in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/packaging-25.0-py314.egg (from transformers)
Requirement already satisfied: filelock in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/filelock-3.16.1-py314.egg (from transformers)
Requirement already satisfied: pyyaml>=5.1 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/pyyaml-6.0.2-py314.egg (from transformers)
Requirement already satisfied: regex!=2019.12.17 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/regex-2024.11.10-cp314-cp314-macosx_11_0_arm64.whl (from transformers)
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/urllib3-2.3.0-py314.egg (from transformers)
```

```
Requirement already satisfied: certifi>=2017.4.17 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib,
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda,
Requirement already satisfied: pytz>=2020.1 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14,
Requirement already satisfied: tzdata>=2022.7 in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14,
Requirement already satisfied: IPython in /Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14,
```

Import Packages

```
#@title Import Packages
```

```
import base64
import functools
import gzip
import json
import time

import einops
import torch
from datasets import load_dataset
from torch import nn
from torch.nn import functional as F
from transformers import AutoTokenizer, DataCollatorForSeq2Seq, Seq2SeqTrainer, Seq2SeqTrainingArguments
from transformers.modeling_outputs import Seq2SeqLMOutput
```

```
TO_SAVE = {"time": time.time()}
```

```
/Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/tqdm/auto.py:21: TqdmWarning:
from .autonotebook import tqdm as notebook_tqdm
```

Data Processing

We'll use the **XSum** dataset (<https://arxiv.org/abs/1808.08745>) to train and evaluate our Transformer model. Each example in the XSum dataset consists of a document and its corresponding summary. Below are the dataset statistics and a sample example:

```
xsum = load_dataset("xsum", trust_remote_code=True, download_mode="force_redownload")
xsum
```

```
Downloading data: 100%|██████████| 2.72M/2.72M [00:00<00:00, 37.0MB/s]
Generating train split: 100%|██████████| 204045/204045 [00:11<00:00, 18110.19 examples/s]
Generating validation split: 100%|██████████| 11332/11332 [00:06<00:00, 1724.18 examples/s]
Generating test split: 100%|██████████| 11334/11334 [00:06<00:00, 1770.48 examples/s]
DatasetDict({
  train: Dataset({
    features: ['document', 'summary', 'id'],
    num_rows: 204045
  })
  validation: Dataset({
    features: ['document', 'summary', 'id'],
    num_rows: 11332
  })
  test: Dataset({
    features: ['document', 'summary', 'id'],
    num_rows: 11334
  })
})
```

```
xsum['train'][0]
```

```
{'document': 'The full cost of damage in Newton Stewart, one of the areas worst affected, is still being assessed.\nRepair work is ongoing in Hawick and many roads in Peeblesshire remain badly affected by standing water.\nTrains on the west coast mainline face disruption due to damage at the Lamington Viaduct.\nMany businesses and householders were affected by flooding in Newton Stewart after the River Cree overflowed into the town.\nFirst Minister Nicola Sturgeon visited the area to inspect the damage.\nThe waters breached a retaining wall, flooding many commercial properties on Victoria Street - the main shopping thoroughfare.\nJeanette Tate, who owns the Cinnamon Cafe which was badly affected, said she could not fault the multi-agency response once the flood hit.\nHowever, she said more preventative work could have been carried out to ensure the retaining wall did not fail.\nIt is difficult but I do think there is so much publicity for Dumfries and the Nith - and I totally appreciate that - but it is almost like we're neglected or forgotten," she said.\nThat may not be true but it is perhaps my perspective over the last few days.\nWhy were you not ready to help us a bit more when the warning and the alarm alerts had gone out?"\nMeanwhile, a flood alert remains in place across the Borders because of the constant rain.\nPeebles was badly hit by problems, sparking calls to introduce more defences in the area.\nScottish Borders Council has put a list on its website of the roads worst affected and drivers have been urged not to ignore closure signs.\nThe Labour Party's deputy Scottish leader Alex Rowley was in Hawick on Monday to see the situation first hand.\nHe said it was important to get the flood protection plan right but backed calls to speed up the process.\nI was quite taken aback by the amount of damage that has been done," he said.\nObviously it is heart-
```

```
breaking for people who have been forced out of their homes and the impact on businesses."
He said it was important that "immediate steps" were taken to protect the areas most vulnerable and a clear timetable put in place for flood prevention plans.
Have you been affected by flooding in Dumfries and Galloway or the Borders? Tell us about your experience of the situation and how it was handled. Email us on selkirk.news@bbc.co.uk or dumfries@bbc.co.uk.,
'summary': 'Clean-up operations are continuing across the Scottish Borders and Dumfries and Galloway after flooding caused by Storm Frank.',
'id': '35232142'}
```

Due to the original XSum dataset's large size, we'll use a subset for training and evaluation that primarily contains science and technology-related news and is about 10% as large as the original dataset. The following code cells will extract this subset for you.

> Binary Blob of Subset Indices

Show code

```
train_indices, validation_indices, test_indices = json.loads(gzip.decompress(base64.b64decode(blob)).decode("utf-8"))
```

```
dataset_train = xsum["train"].select(train_indices)
dataset_validation = xsum["validation"].select(validation_indices)
dataset_test = xsum["test"].select(test_indices)
```

```
dataset_train, dataset_validation, dataset_test
```

```
(Dataset({
  features: ['document', 'summary', 'id'],
  num_rows: 24350
}),
Dataset({
  features: ['document', 'summary', 'id'],
  num_rows: 1281
}),
Dataset({
  features: ['document', 'summary', 'id'],
  num_rows: 1326
}))
```

✓ Tokenization and Collating

Before feeding the data into the model, we must convert raw texts into sequences of indices. Since our focus isn't on NLP specifics, we'll skip tokenization details and directly reuse the tokenizer and vocabulary from a pre-trained T5 model. The `tokenize` function tokenizes both the document and summary of each example before adding them to the data structure.

```
tokenizer = AutoTokenizer.from_pretrained("t5-small")
```

```
def tokenize(tokenizer, examples):
    inputs = [doc for doc in examples["document"]]
    model_inputs = tokenizer(inputs, max_length=1024, truncation=True)
    labels = tokenizer(examples["summary"], max_length=128, truncation=True)
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs
```

```
tokenize_fn = functools.partial(tokenize, tokenizer)
```

```
dataset_train = dataset_train.map(tokenize_fn, batched=True)
dataset_validation = dataset_validation.map(tokenize_fn, batched=True)
dataset_test = dataset_test.map(tokenize_fn, batched=True)
```

```
Parameter 'fn_kwargs'={} of the transform datasets.arrow_dataset.Dataset._map_single couldn't be hashed properly, a r
Map: 100%|██████████| 24350/24350 [00:09<00:00, 2664.01 examples/s]
Map: 100%|██████████| 1281/1281 [00:00<00:00, 3393.05 examples/s]
Map: 100%|██████████| 1326/1326 [00:00<00:00, 3260.30 examples/s]
```

```
dataset_train[0].keys()
```

```
dict_keys(['document', 'summary', 'id', 'input_ids', 'attention_mask', 'labels'])
```

During training, tokenized examples are collated into batches before being fed into the model. We'll use `DataCollatorForSeq2Seq` from the `transformers` library, which conveniently handles padding and tensor conversions of both inputs and outputs for us.


```
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model="t5-small")
```

✓ Implement Transformer

In this section, you'll implement the Transformer step by step. Starting with scaled dot-product attention and multi-head attention, you'll progress to Transformer layers and ultimately the Transformer encoder-decoder model. Please refer to the "Attention is All You Need" paper for implementation specifics.

✓ Scaled Dot-Product Attention

In this section, you'll implement Scaled Dot-Product Attention. The input types and shapes are:

- `q`: `Tensor[n, tgt_len, d_head]`
- `k`: `Tensor[n, src_len, d_head]`
- `v`: `Tensor[n, src_len, d_head]`
- `key_padding_mask`: `Tensor[n, src_len]`
- `causal`: `bool`

`n` represents the total number of attention operations calculated in parallel. In multi-head attention, it's usually the product of the batch size and the number of attention heads. For each of the `n` operations, compute attention scores

$$s_{i,j} = \mathbf{q}_i^T \mathbf{k}_j / \sqrt{d_{\text{head}}}$$

Apply softmax to the attention score, then use it as weights to linearly combine values in `v`:

$$a_{i,j} = \frac{\exp(s_{i,j})}{\sum_k \exp(s_{i,k})}$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})_i = \sum_j a_{i,j} \mathbf{v}_j$$

However, consider two essential details: *padding* and *causal masking*.

- *padding*: The Transformer input usually contains sentences of varying lengths, padded into a tensor. Attention should ignore pad tokens, so they don't impact the results. `key_padding_mask` is a byte tensor set to `1` in pad token positions. If `key_padding_mask` is `None`, there's no padding in the input.
- *causal masking*: Autoregressive generation in the decoder uses causal attention masks, meaning position i can only attend to position j if $i \geq j$. If `causal` is set to true, apply causal attention masking. The provided `future_mask` may be useful.

Implement scaled dot-product attention below. Ensure your implementation is fully vectorized (no for-loops allowed). Avoid using `torch.nn.functional.multi_head_attention_forward` or any classes in `torch.nn`.

Hint: Consider using `einops` (<https://einops.rocks/>).

Hint: Alternatively, `torch.bmm` or `torch.einsum` may be helpful.

Hint: To mask an element in the attention map, set its pre-attention score to `-inf`, ensuring its post-attention score is always 0.

```
future_mask = torch.triu(torch.zeros([1024, 1024]).fill_(float("-inf")), 1)
future_mask
```

```
tensor([[0., -inf, -inf, ..., -inf, -inf, -inf],
        [0., 0., -inf, ..., -inf, -inf, -inf],
        [0., 0., 0., ..., -inf, -inf, -inf],
        ...,
        [0., 0., 0., ..., 0., -inf, -inf],
        [0., 0., 0., ..., 0., 0., -inf],
        [0., 0., 0., ..., 0., 0., 0.]])
```

```
def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False):
    #####
    # TODO: implement this function
    #####
    # q: [n, tgt_len, d_head]
    # k: [n, src_len, d_head]
    # v: [n, src_len, d_head]
    # key_padding_mask: [n, src_len] - byte tensor with 1 at pad positions
    # causal: bool - if True, apply causal masking
```

```

n, tgt_len, d_head = q.shape
src_len = k.shape[1]

# Compute attention scores: s_{i,j} = q_i^T k_j / sqrt(d_head)
# Using einsum: [n, tgt_len, d_head] x [n, src_len, d_head] -> [n, tgt_len, src_len]
scores = torch.einsum('ntd,nsd->nts', q, k) / (d_head ** 0.5)

# Apply key padding mask: set pad positions to -inf
if key_padding_mask is not None:
    # key_padding_mask: [n, src_len] -> expand to [n, 1, src_len] for broadcasting
    mask = key_padding_mask.unsqueeze(1).bool() # [n, 1, src_len]
    scores = scores.masked_fill(mask, float('-inf'))

# Apply causal mask: position i can only attend to positions j <= i
if causal:
    # Use future_mask which has -inf in upper triangular positions
    causal_mask = future_mask[:tgt_len, :src_len].to(scores.device)
    scores = scores + causal_mask

# Apply softmax to get attention weights (convert to float32 for numerical stability)
attn_weights = F.softmax(scores.float(), dim=-1).type_as(scores)

# Apply attention weights to values
# [n, tgt_len, src_len] x [n, src_len, d_head] -> [n, tgt_len, d_head]
output = torch.einsum('nts,nsd->ntd', attn_weights, v)

return output
#####

```

Notes: The implementation for the dot product attention resembles that of the answer

Ensure your code passes the following tests:

```

def test_scaled_dot_product_attention():
    q1 = torch.tensor([[1, 0, 0], [0.5, 0.5, 0]]).view(1, 2, 3).float()
    k1 = torch.tensor([[1, 0, 0], [0, 1, 0], [0, 0, 1]]).view(1, 3, 3).float()
    v1 = torch.tensor([[3, 0, 0], [0, 5, 0], [0, 0, 7]]).view(1, 3, 3).float()
    o1 = scaled_dot_product_attention(q1, k1, v1)
    assert list(o1.shape) == [1, 2, 3]
    assert torch.allclose(
        o1.view(-1)[:5],
        torch.tensor([1.413249135017395, 1.3222922086715698, 1.8512091636657715, 1.0912044048309326, 1.8186740875244
            rtol=1e-3
    )

    torch.manual_seed(100)
    q2 = torch.randn(3, 5, 7).float()
    k2 = torch.randn(3, 11, 7).float()
    v2 = torch.randn(3, 11, 7).float()
    o2 = scaled_dot_product_attention(q2, k2, v2)
    assert list(o2.shape) == [3, 5, 7]
    assert torch.allclose(
        o2.view(-1)[6: 11],
        torch.tensor([-0.40304261445999146, -0.2931785583496094, 0.20563912391662598, 0.08719107508659363, 0.0827403
            rtol=1e-3
    )

    torch.manual_seed(200)
    q3 = torch.randn(7, 5, 6).float()
    k3 = torch.randn(7, 3, 6).float()
    v3 = torch.randn(7, 3, 6).float()
    o3 = scaled_dot_product_attention(q3, k3, v3)
    T0_SAVE["scaled_dot_product_attention.3.shape"] = list(o3.shape)
    T0_SAVE["scaled_dot_product_attention.3.value"] = o3.view(-1)[5: 10].tolist()

    key_padding_mask = torch.tensor([[0, 0, 1]]).byte()
    o4 = scaled_dot_product_attention(q1, k1, v1, key_padding_mask=key_padding_mask)
    assert list(o4.shape) == [1, 2, 3]
    assert torch.allclose(
        o4.view(-1)[:5],
        torch.tensor([1.921372413635254, 1.7977124452590942, 0.0, 1.5, 2.5]),
        rtol=1e-3
    )

    torch.manual_seed(210)

```

```

q5 = torch.randn(2, 4, 3).float()
k5 = torch.randn(2, 4, 3).float()
v5 = torch.randn(2, 4, 3).float()
o5 = scaled_dot_product_attention(q5, k5, v5, causal=True)
assert list(o5.shape) == [2, 4, 3]
assert torch.allclose(
    o5.view(-1)[2: 7],
    torch.tensor([0.9079901576042175, -0.573272705078125, -1.1765587329864502, 0.7771514058113098, -0.3235766291
    rtol=1e-3
)

torch.manual_seed(220)
q6 = torch.randn(3, 5, 4).float()
k6 = torch.randn(3, 5, 4).float()
v6 = torch.randn(3, 5, 4).float()
key_padding_mask = torch.tensor([
    [0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0],
    [0, 0, 0, 1, 1]
]).byte()
o6 = scaled_dot_product_attention(q6, k6, v6, key_padding_mask=key_padding_mask, causal=True)
T0_SAVE["scaled_dot_product_attention.6.shape"] = list(o6.shape)
T0_SAVE["scaled_dot_product_attention.6.value"] = o6.view(-1)[3: 8].tolist()

test_scaled_dot_product_attention()

```

Notes: Code is correct as the tests pass!

Multi-head Attention

In this section, you'll implement multi-head attention.

The input to this layer has types and shapes:

- `q`: `Tensor[bsz, tgt_len, d_model]`
- `k`: `Tensor[bsz, src_len, d_model]`
- `v`: `Tensor[bsz, src_len, d_model]`
- `key_padding_mask`: `Tensor[bsz, src_len]`
- `causal`: `bool`

A multi-head attention layer has four linear projection layers (including biases in this assignment): `q_proj`, `k_proj`, `v_proj`, and `o_proj`. `q_proj`, `k_proj`, and `v_proj` project `q`, `k`, and `v` respectively into `n_heads` `d_head` vectors. The shapes of the projected query, key, and value will be `[bsz, tgt_len, n_heads, d_head]`, `[bsz, src_len, n_heads, d_head]`, and `[bsz, src_len, n_heads, d_head]` respectively.

In the provided code below, instead of creating `n_heads` projection matrices of `d_model -> d_head` for the query/key/value, we use a single projection matrix of `d_model -> d_model`. This means the first `d_head` channels correspond to the first attention head, and channels from `d_head + 1` to `2 * d_head` correspond to the second attention head, and so on. The same rule applies to the input channels of `o_proj`.

Next, rearrange the projected query, key, and value tensors appropriately and feed them into your previously implemented `scaled_dot_product_attention` function.

The output of `scaled_dot_product_attention` should then be projected by `o_proj` to produce the final output. The output shape should be `[bsz, tgt_len, d_model]`.

Implement multi-head attention below. Avoid using `nn.MultiheadAttention` or `torch.nn.functional.multi_head_attention_forward`. Ensure your implementation is fully vectorized.

```

class MultiheadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.o_proj = nn.Linear(d_model, d_model)
        self.n_heads = n_heads
        self.d_head = d_model // n_heads

    def forward(self, q, k, v, key_padding_mask=None, causal=False):

```

```
#####
# TODO: implement this method
#####
# q: [bsz, tgt_len, d_model]
# k: [bsz, src_len, d_model]
# v: [bsz, src_len, d_model]
# key_padding_mask: [bsz, src_len]

bsz, tgt_len, d_model = q.shape
src_len = k.shape[1]

# Project q, k, v
q = self.q_proj(q) # [bsz, tgt_len, d_model]
k = self.k_proj(k) # [bsz, src_len, d_model]
v = self.v_proj(v) # [bsz, src_len, d_model]

# Reshape to [bsz, seq_len, n_heads, d_head] then rearrange to [bsz * n_heads, seq_len, d_head]
# Using einops for clarity
q = einops.rearrange(q, 'b t (h d) -> (b h) t d', h=self.n_heads, d=self.d_head)
k = einops.rearrange(k, 'b s (h d) -> (b h) s d', h=self.n_heads, d=self.d_head)
v = einops.rearrange(v, 'b s (h d) -> (b h) s d', h=self.n_heads, d=self.d_head)

# Expand key_padding_mask for n_heads
if key_padding_mask is not None:
    # [bsz, src_len] -> [bsz * n_heads, src_len]
    key_padding_mask = einops.repeat(key_padding_mask, 'b s -> (b h) s', h=self.n_heads)

# Apply scaled dot-product attention
attn_output = scaled_dot_product_attention(q, k, v, key_padding_mask, causal)
# attn_output: [bsz * n_heads, tgt_len, d_head]

# Reshape back to [bsz, tgt_len, d_model]
attn_output = einops.rearrange(attn_output, '(b h) t d -> b t (h d)', b=bsz, h=self.n_heads)

# Final projection
output = self.o_proj(attn_output) # [bsz, tgt_len, d_model]

return output
#####
```

Notes: Cursor is able to replicate the code exactly

The following code checks your implementation against `nn.MultiheadAttention` in PyTorch:

```
def test_multihead_attention():
    torch.manual_seed(350)
    mha0 = nn.MultiheadAttention(embed_dim=128, num_heads=4, batch_first=True)
    nn.init.normal_(mha0.in_proj_weight, mean=0.0, std=0.05)
    nn.init.normal_(mha0.in_proj_bias, mean=0.0, std=0.05)
    nn.init.normal_(mha0.out_proj.weight, mean=0.0, std=0.05)
    nn.init.normal_(mha0.out_proj.bias, mean=0.0, std=0.05)
    mha1 = MultiheadAttention(128, 4)
    mha1.q_proj.weight.data.copy_(mha0.in_proj_weight.data[:128, :])
    mha1.q_proj.bias.data.copy_(mha0.in_proj_bias.data[:128])
    mha1.k_proj.weight.data.copy_(mha0.in_proj_weight.data[128:256, :])
    mha1.k_proj.bias.data.copy_(mha0.in_proj_bias.data[128:256])
    mha1.v_proj.weight.data.copy_(mha0.in_proj_weight.data[256:, :])
    mha1.v_proj.bias.data.copy_(mha0.in_proj_bias.data[256:])
    mha1.o_proj.weight.data.copy_(mha0.out_proj.weight.data)
    mha1.o_proj.bias.data.copy_(mha0.out_proj.bias.data)

    torch.manual_seed(400)
    q1 = torch.randn(4, 6, 128).float()
    k1 = torch.randn(4, 6, 128).float()
    v1 = torch.randn(4, 6, 128).float()
    assert torch.allclose(
        mha0(q1, k1, v1)[0].contiguous(),
        mha1(q1, k1, v1).contiguous(),
        rtol=1e-3
    )

    torch.manual_seed(500)
    q2 = torch.randn(2, 5, 128).float()
    k2 = torch.randn(2, 3, 128).float()
    v2 = torch.randn(2, 3, 128).float()
```

```

o20 = mha0(q2, k2, v2)[0].contiguous()
o21 = mha1(q2, k2, v2).contiguous()
TO_SAVE["multihead_attention.2.0.shape"] = list(o20.shape)
TO_SAVE["multihead_attention.2.0.value"] = o20.view(-1)[126: 131].tolist()
TO_SAVE["multihead_attention.2.1.shape"] = list(o21.shape)
TO_SAVE["multihead_attention.2.1.value"] = o21.view(-1)[126: 131].tolist()

torch.manual_seed(600)
q3 = torch.randn(4, 6, 128).float()
k3 = torch.randn(4, 6, 128).float()
v3 = torch.randn(4, 6, 128).float()
key_padding_mask = torch.tensor([
    [0, 0, 1, 1, 1, 1],
    [0, 0, 0, 0, 1, 1],
    [0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 1]
]).byte()
o30 = mha0(
    q3, k3, v3,
    key_padding_mask=key_padding_mask.to(torch.bool),
    attn_mask=future_mask[:6, :6]
)[0].contiguous()
o31 = mha1(q3, k3, v3, key_padding_mask=key_padding_mask, causal=True).contiguous()
assert torch.allclose(o30[0, :2], o31[0, :2], rtol=1e-3)
assert torch.allclose(o30[0, :4], o31[0, :4], rtol=1e-3)
assert torch.allclose(o30[0, :6], o31[0, :6], rtol=1e-3)
assert torch.allclose(o30[0, :5], o31[0, :5], rtol=1e-3)

torch.manual_seed(700)
q4 = torch.randn(2, 5, 128).float()
k4 = torch.randn(2, 5, 128).float()
v4 = torch.randn(2, 5, 128).float()
key_padding_mask = torch.tensor([
    [0, 0, 1, 1, 1],
    [0, 0, 0, 0, 1],
]).byte()
o40 = mha0(
    q4, k4, v4,
    key_padding_mask=key_padding_mask.to(torch.bool),
    attn_mask=future_mask[:5, :5]
)[0].contiguous()
o41 = mha1(q4, k4, v4, key_padding_mask=key_padding_mask, causal=True).contiguous()
TO_SAVE["multihead_attention.4.0.shape"] = list(o40.shape)
TO_SAVE["multihead_attention.4.0.value"] = o40.view(-1)[126: 131].tolist()
TO_SAVE["multihead_attention.4.1.shape"] = list(o41.shape)
TO_SAVE["multihead_attention.4.1.value"] = o41.view(-1)[126: 131].tolist()

test_multihead_attention()

/Users/arnavdalal/Documents/fa2025/eecs182/hw10code/.conda/lib/python3.14/site-packages/torch/nn/functional.py:6044:
warnings.warn(

```

Notes: The implementation passes the tests zero-shot!

✓ Transformer Layers

In this section, you'll **implement Transformer Encoder/Decoder layers** according to Figure 1 of "Attention is All You Need". Note that you should also apply residual dropout (Section 5.4 of the paper). Activation dropout and attention dropout will not be used in this assignment.

The `is_decoder` flag determines if this Transformer layer is an encoder or a decoder.

If it's an encoder, the input types and shapes will be:

- `x`: `Tensor[bsz, src_len, d_model]`
- `padding_mask`: `Tensor[bsz, src_len]`

If it's a decoder, the input types and shapes will be:

- `x`: `Tensor[bsz, tgt_len, d_model]`
- `padding_mask`: `Tensor[bsz, tgt_len]`
- `encoder_out`: `Tensor[bsz, src_len, d_model]`
- `encoder_padding_mask`: `Tensor[bsz, src_len]`

The output should be a tensor of the same shape as `x`.

Avoid using `nn.TransformerEncoderLayer` or `nn.TransformerDecoderLayer`.

```
class TransformerLayer(nn.Module):
    def __init__(self, is_decoder, d_model, n_heads, d_ffn, p_drop):
        super().__init__()
        self.is_decoder = is_decoder
        self.self_attn = MultiheadAttention(d_model, n_heads)
        self.self_attn_drop = nn.Dropout(p_drop)
        self.self_attn_ln = nn.LayerNorm(d_model)
        if is_decoder:
            self.cross_attn = MultiheadAttention(d_model, n_heads)
            self.cross_attn_drop = nn.Dropout(p_drop)
            self.cross_attn_ln = nn.LayerNorm(d_model)
        self.fc1 = nn.Linear(d_model, d_ffn)
        self.fc2 = nn.Linear(d_ffn, d_model)
        self.ffn_drop = nn.Dropout(p_drop)
        self.ffn_ln = nn.LayerNorm(d_model)

    def forward(self, x, padding_mask, encoder_out=None, encoder_padding_mask=None):
        #####
        # TODO: implement this method
        #####
        # For encoder: x: [bsz, src_len, d_model], padding_mask: [bsz, src_len]
        # For decoder: x: [bsz, tgt_len, d_model], padding_mask: [bsz, tgt_len]
        # encoder_out: [bsz, src_len, d_model], encoder_padding_mask: [bsz, src_len]

        # Self-attention (with causal masking for decoder)
        residual = x
        if self.is_decoder:
            # Decoder self-attention with causal mask
            x = self.self_attn(x, x, x, key_padding_mask=padding_mask, causal=True)
        else:
            # Encoder self-attention (no causal mask)
            x = self.self_attn(x, x, x, key_padding_mask=padding_mask, causal=False)
        x = self.self_attn_drop(x)
        x = self.self_attn_ln(residual + x)

        # Cross-attention (only for decoder)
        if self.is_decoder:
            residual = x
            x = self.cross_attn(x, encoder_out, encoder_out, key_padding_mask=encoder_padding_mask, causal=False)
            x = self.cross_attn_drop(x)
            x = self.cross_attn_ln(residual + x)

        # Feed-forward network
        residual = x
        x = self.fc2(F.relu(self.fc1(x)))
        x = self.ffn_drop(x)
        x = self.ffn_ln(residual + x)

        return x
        #####
```

Notes: The structure for this implementation is slightly different than the answer key, but the functionality is the same.

The following code checks your implementation against `nn.TransformerEncoderLayer` and `nn.TransformerDecoderLayer` in PyTorch:

```
def test_transformer_layer():
    torch.manual_seed(750)
    enc_layer0 = nn.TransformerEncoderLayer(128, 4, dim_feedforward=512, dropout=0.0, batch_first=True)
    nn.init.normal_(enc_layer0.self_attn.in_proj_weight, mean=0.0, std=0.05)
    nn.init.normal_(enc_layer0.self_attn.in_proj_bias, mean=0.0, std=0.05)
    nn.init.normal_(enc_layer0.self_attn.out_proj.weight, mean=0.0, std=0.05)
    nn.init.normal_(enc_layer0.self_attn.out_proj.bias, mean=0.0, std=0.05)
    nn.init.normal_(enc_layer0.linear1.weight, mean=0.0, std=0.05)
    nn.init.normal_(enc_layer0.linear1.bias, mean=0.0, std=0.05)
    nn.init.normal_(enc_layer0.linear2.weight, mean=0.0, std=0.05)
    nn.init.normal_(enc_layer0.linear2.bias, mean=0.0, std=0.05)
    enc_layer1 = TransformerLayer(False, 128, 4, 512, 0.0)
    enc_layer1.self_attn.q_proj.weight.data.copy_(enc_layer0.self_attn.in_proj_weight.data[:128, :])
```

```

enc_layer1.self_attn.q_proj.bias.data.copy_(enc_layer0.self_attn.in_proj_bias.data[:128])
enc_layer1.self_attn.k_proj.weight.data.copy_(enc_layer0.self_attn.in_proj_weight.data[128:256, :])
enc_layer1.self_attn.k_proj.bias.data.copy_(enc_layer0.self_attn.in_proj_bias.data[128:256])
enc_layer1.self_attn.v_proj.weight.data.copy_(enc_layer0.self_attn.in_proj_weight.data[256:, :])
enc_layer1.self_attn.v_proj.bias.data.copy_(enc_layer0.self_attn.in_proj_bias.data[256:])
enc_layer1.self_attn.o_proj.weight.data.copy_(enc_layer0.self_attn.out_proj.weight.data)
enc_layer1.self_attn.o_proj.bias.data.copy_(enc_layer0.self_attn.out_proj.bias.data)
enc_layer1.fc1.weight.data.copy_(enc_layer0.linear1.weight.data)
enc_layer1.fc1.bias.data.copy_(enc_layer0.linear1.bias.data)
enc_layer1.fc2.weight.data.copy_(enc_layer0.linear2.weight.data)
enc_layer1.fc2.bias.data.copy_(enc_layer0.linear2.bias.data)

torch.manual_seed(800)
x = torch.randn(4, 5, 128).float()
x_mask = torch.tensor([[0, 0, 0, 0, 0], [0, 0, 0, 1, 1], [0, 0, 0, 0, 1], [0, 0, 1, 1, 1]]).byte()
y10 = enc_layer0(x, src_key_padding_mask=x_mask.to(torch.bool)).contiguous()
y11 = enc_layer1(x, x_mask).contiguous()
assert torch.allclose(y10[0], y11[0], rtol=1e-3)
assert torch.allclose(y10[1, :3], y11[1, :3], rtol=1e-3)
assert torch.allclose(y10[2, :4], y11[2, :4], rtol=1e-3)
assert torch.allclose(y10[3, :2], y11[3, :2], rtol=1e-3)

torch.manual_seed(900)
x = torch.randn(3, 4, 128).float()
x_mask = torch.tensor([[0, 0, 0, 1], [0, 0, 0, 1], [0, 0, 1, 1]]).byte()
y20 = enc_layer0(x, src_key_padding_mask=x_mask.to(torch.bool)).contiguous()
y21 = enc_layer1(x, x_mask).contiguous()
T0_SAVE["transformer_layer.2.0.shape"] = list(y20.shape)
T0_SAVE["transformer_layer.2.0.value"] = y21.view(-1)[126: 131].tolist()
T0_SAVE["transformer_layer.2.1.shape"] = list(y20.shape)
T0_SAVE["transformer_layer.2.1.value"] = y21.view(-1)[126: 131].tolist()

torch.manual_seed(950)
dec_layer0 = nn.TransformerDecoderLayer(128, 4, dim_feedforward=512, dropout=0.0, batch_first=True)
nn.init.normal_(dec_layer0.self_attn.in_proj_weight, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.self_attn.in_proj_bias, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.self_attn.out_proj.weight, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.self_attn.out_proj.bias, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.multihead_attn.in_proj_weight, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.multihead_attn.in_proj_bias, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.multihead_attn.out_proj.weight, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.multihead_attn.out_proj.bias, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.linear1.weight, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.linear1.bias, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.linear2.weight, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.linear2.bias, mean=0.0, std=0.05)
dec_layer1 = TransformerLayer(True, 128, 4, 512, 0.0)
dec_layer1.self_attn.q_proj.weight.data.copy_(dec_layer0.self_attn.in_proj_weight.data[:128, :])
dec_layer1.self_attn.q_proj.bias.data.copy_(dec_layer0.self_attn.in_proj_bias.data[:128])
dec_layer1.self_attn.k_proj.weight.data.copy_(dec_layer0.self_attn.in_proj_weight.data[128:256, :])
dec_layer1.self_attn.k_proj.bias.data.copy_(dec_layer0.self_attn.in_proj_bias.data[128:256])
dec_layer1.self_attn.v_proj.weight.data.copy_(dec_layer0.self_attn.in_proj_weight.data[256:, :])
dec_layer1.self_attn.v_proj.bias.data.copy_(dec_layer0.self_attn.in_proj_bias.data[256:])
dec_layer1.self_attn.o_proj.weight.data.copy_(dec_layer0.self_attn.out_proj.weight.data)
dec_layer1.self_attn.o_proj.bias.data.copy_(dec_layer0.self_attn.out_proj.bias.data)
dec_layer1.cross_attn.q_proj.weight.data.copy_(dec_layer0.multihead_attn.in_proj_weight.data[:128, :])
dec_layer1.cross_attn.q_proj.bias.data.copy_(dec_layer0.multihead_attn.in_proj_bias.data[:128])
dec_layer1.cross_attn.k_proj.weight.data.copy_(dec_layer0.multihead_attn.in_proj_weight.data[128:256, :])
dec_layer1.cross_attn.k_proj.bias.data.copy_(dec_layer0.multihead_attn.in_proj_bias.data[128:256])
dec_layer1.cross_attn.v_proj.weight.data.copy_(dec_layer0.multihead_attn.in_proj_weight.data[256:, :])
dec_layer1.cross_attn.v_proj.bias.data.copy_(dec_layer0.multihead_attn.in_proj_bias.data[256:])
dec_layer1.cross_attn.o_proj.weight.data.copy_(dec_layer0.multihead_attn.out_proj.weight.data)
dec_layer1.cross_attn.o_proj.bias.data.copy_(dec_layer0.multihead_attn.out_proj.bias.data)
dec_layer1.fc1.weight.data.copy_(dec_layer0.linear1.weight.data)
dec_layer1.fc1.bias.data.copy_(dec_layer0.linear1.bias.data)
dec_layer1.fc2.weight.data.copy_(dec_layer0.linear2.weight.data)
dec_layer1.fc2.bias.data.copy_(dec_layer0.linear2.bias.data)

torch.manual_seed(1000)
x = torch.randn(4, 5, 128).float()
e = torch.randn(4, 3, 128).float()
x_mask = torch.tensor([[0, 0, 0, 0, 0], [0, 0, 0, 1, 1], [0, 0, 0, 0, 1], [0, 0, 1, 1, 1]]).byte()
e_mask = torch.tensor([[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 1]]).byte()
y30 = dec_layer0(x, e, tgt_mask=future_mask[:5, :5], tgt_key_padding_mask=x_mask.to(torch.bool), memory_key_padd
y31 = dec_layer1(x, x_mask, e, e_mask).contiguous()
assert torch.allclose(y30[0], y31[0], rtol=1e-3)
assert torch.allclose(y30[1, :3], y31[1, :3], rtol=1e-3)

```

```

assert torch.allclose(y30[2, :4], y31[2, :4], rtol=1e-3)
assert torch.allclose(y30[3, :2], y31[3, :2], rtol=1e-3)

torch.manual_seed(1100)
x = torch.randn(3, 4, 128).float()
e = torch.randn(3, 3, 128).float()
x_mask = torch.tensor([[0, 0, 0, 0], [0, 0, 0, 1], [0, 0, 1, 1]]).byte()
e_mask = torch.tensor([[0, 0, 0], [0, 0, 0], [0, 0, 1]]).byte()
y40 = dec_layer0(x, e, tgt_mask=future_mask[:4, :4], tgt_key_padding_mask=x_mask.to(torch.bool), memory_key_padd
y41 = dec_layer1(x, x_mask, e, e_mask).contiguous()
T0_SAVE["transformer_layer.4.0.shape"] = list(y40.shape)
T0_SAVE["transformer_layer.4.0.value"] = y41.view(-1)[126: 131].tolist()
T0_SAVE["transformer_layer.4.1.shape"] = list(y40.shape)
T0_SAVE["transformer_layer.4.1.value"] = y41.view(-1)[126: 131].tolist()

test_transformer_layer()

```

Notes: Cursor completed this test zero-shot as well.

✓ Putting them together: Transformer

In this section, you will implement a Transformer encoder-decoder model for sequence-to-sequence tasks using the building blocks you've already created: scaled dot-product attention, multi-head attention, and Transformer encoder/decoder layers.

Model Overview:

- Encoder: `n_layers` layers
- Decoder: `n_layers` layers
- Learned positional embeddings instead of sinusoidal positional encodings (as in "Attention is All You Need")
- Shared embedding matrices to reduced number of parameters and to improve training stability:
 - Encoder input embeddings
 - Decoder input embeddings
 - Decoder output layer weights (a linear classifier over `n_words` vocabulary, whose weight matrix happens to have the same shape as word embeddings, so their weights can be shared)
- Layer normalization after the embedding layers
- Shared positional embedding matrix and normalization layer for both encoder and decoder
- Decoder's first input token: `[EOS]`
- Handling of pad tokens in labels (-100). Huggingface `transformers` pads labels with padding index -100 instead of `pad_id`. So we do processing as follows:
 - Replace -100 in decoder input
 - Ignore -100 in labels when calculating loss

Implement the `make_positions` method to generate input for the positional embedding layer. The output of `make_positions` should be a tensor with the same dtype and shape as `input_ids`. For example, if the input is a batch of 2 sequences with a length of 5, the output of `make_positions` should be:

```

[[0, 1, 2, 3, 4],
 [0, 1, 2, 3, 4]]

```

```

class Transformer(nn.Module):
    def __init__(self, n_words, pad_id, eos_id, max_len, n_layers, d_model, n_heads, d_ffn, p_drop):
        super().__init__()
        self.pad_id = pad_id
        self.eos_id = eos_id
        self.emb_word = nn.Embedding(n_words, d_model)
        self.emb_pos = nn.Embedding(max_len, d_model)
        self.emb_word.weight.data.uniform_(-0.05, 0.05)
        self.emb_pos.weight.data.uniform_(-0.05, 0.05)
        self.emb_ln = nn.LayerNorm(d_model)
        self.encoder_layers = nn.ModuleList([
            TransformerLayer(False, d_model, n_heads, d_ffn, p_drop)
            for _ in range(n_layers)
        ])
        self.decoder_layers = nn.ModuleList([
            TransformerLayer(True, d_model, n_heads, d_ffn, p_drop)
            for _ in range(n_layers)
        ])

```



```

    ))
    self.lm_head = nn.Linear(d_model, n_words)
    self.lm_head.weight = self.emb_word.weight
    self.criterion = nn.CrossEntropyLoss(ignore_index=-100)

def make_positions(self, input_ids, padding_mask):
    #####
    # TODO: implement this method
    #####
    # input_ids: [bsz, seq_len]
    # padding_mask: [bsz, seq_len] (not used here, but provided for reference)
    # Returns: [bsz, seq_len] tensor of position indices

    bsz, seq_len = input_ids.shape
    # Create position indices [0, 1, 2, ..., seq_len-1] and expand to batch size
    positions = torch.arange(seq_len, dtype=input_ids.dtype, device=input_ids.device)
    positions = positions.unsqueeze(0).expand(bsz, -1)
    return positions
    #####

def forward(self, input_ids, attention_mask, labels):
    enc_padding_mask = input_ids.eq(self.pad_id).byte()
    enc_pos = self.make_positions(input_ids, enc_padding_mask)
    enc_state = self.emb_ln(self.emb_word(input_ids) + self.emb_pos(enc_pos))
    for layer in self.encoder_layers:
        enc_state = layer(enc_state, enc_padding_mask)
    decoder_input_ids = labels.new_zeros(labels.shape)
    decoder_input_ids[:, 1:] = labels[:, :-1].clone()
    decoder_input_ids[:, 0] = self.eos_id
    decoder_input_ids.masked_fill_(decoder_input_ids == -100, self.pad_id)
    dec_padding_mask = decoder_input_ids.eq(self.pad_id).byte()
    dec_pos = self.make_positions(decoder_input_ids, dec_padding_mask)
    dec_state = self.emb_ln(self.emb_word(decoder_input_ids) + self.emb_pos(dec_pos))
    for layer in self.decoder_layers:
        dec_state = layer(dec_state, dec_padding_mask, enc_state, enc_padding_mask)
    lm_logits = self.lm_head(dec_state)
    loss = self.criterion(lm_logits.view(-1, lm_logits.size(-1)), labels.view(-1))
    return Seq2SeqLMOutput(
        loss=loss,
        logits=lm_logits
    )

```

Notes: The solution that cursor comes up with here is very different from the answer key, which shows an elegant 1 line solution for the same. Either way, the functionality is the same as we see later in this notebook during the training.

✧ Train the Transformer for Summarization

Once the model and the data are both ready, we can begin training our Transformer encoder-decoder model for summarization. In the following code cell, we define a Transformer encoder-decoder model with a 4-layer encoder and a 4-layer decoder, both containing 8 attention heads with a dimension of 64.

```

model = Transformer(
    tokenizer.vocab_size,
    tokenizer.pad_token_id,
    tokenizer.eos_token_id,
    max_len=1024,
    n_layers=4,
    d_model=512,
    n_heads=8,
    d_ffn=2048,
    p_drop=0.1
)

```

We'll start a Tensorboard to visualize the training of the model.

Note: When started in Google Colab, Tensorboard does not default to automatically update, so please click the  symbol on the top-right to manually update the Tensorboard. Or open the `Settings -> Reload data`.

⚠ The following code is a workaround (2023 October), due to some recent update in Google Chrome.

The issue is currently live: <https://github.com/googlecolab/colabtools/issues/3990>.

Hopefully it would not be long before it is fixed.

```
!pip install tensorboard
import tensorboard
```

```
Requirement already satisfied: tensorboard in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: absl-py>=0.4 in /usr/local/lib/python3.12/dist-packages (from tensorboard) (1.4.0)
Requirement already satisfied: grpcio>=1.48.2 in /usr/local/lib/python3.12/dist-packages (from tensorboard) (1.76.0)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard) (3.10)
Requirement already satisfied: numpy>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorboard) (25.0)
Requirement already satisfied: protobuf!=4.24.0,>=3.19.6 in /usr/local/lib/python3.12/dist-packages (from tensorboard) (75.2)
Requirement already satisfied: setuptools>=41.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard) (75.2)
Requirement already satisfied: six>1.9 in /usr/local/lib/python3.12/dist-packages (from tensorboard) (1.17.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard) (0.7.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard) (3.1.3)
Requirement already satisfied: typing-extensions~4.12 in /usr/local/lib/python3.12/dist-packages (from grpcio>=1.48.2) (4.12.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->te
```

```
%load_ext tensorboard
# ignore the error in the output and run the next cell
%tensorboard --logdir my_xsum_model/logs --host=127.0.0.1 --port=6006 --load_fast=false
```

TensorBoard

INACTIVE

No dashboards are active for the current data set.

Probable causes:

- You haven't written any data to your event files.
- TensorBoard can't find your event files.

If you're new to using TensorBoard, and want to find out how to add data and set up your event files, check out the [README](#) and perhaps the [TensorBoard tutorial](#).

If you think TensorBoard is configured properly, please see [the section of the README devoted to missing data problems](#) and consider filing an issue on GitHub.

Last reload: Dec 10, 2025, 10:51:19 PM

Log directory: my_xsum_model/logs

```
from google.colab import output
output.serve_kernel_port_as_window(6006, path='')
# click the link https://localhost:6006
```

Warning: This function may stop working due to changes in browser security.
Try 'serve_kernel_port_as_iframe' instead.

We'll train the model using the following hyperparameters with `Seq2SeqTrainer` from Huggingface's `transformers`. The trainer will automatically move the model to the GPU, set it to training mode, and run forward pass, backward pass, and optimization steps for us. It will also update the Tensorboard you started above (press the refresh button at the top right corner of the Tensorboard page to reload it if it does not update).

Training will take a considerable amount of time (~20 minutes on a Tesla T4 GPU), so you're not required to tune these hyperparameters. Just **ensure that your implementation is correct**. In this case, the validation loss of your model after 2 epochs should be close to **5.0**.

The training runs in FP16 mixed precision mode, which generally speeds up training, especially on newer GPUs (like the ones available if you're using Colab Pro). This is a common practice in modern NLP. However, if your implementation has numerical stability issues, it could cause instability during training. If you encounter precision-related issues, double-check your implementation of scaled dot-product attention, and *ensure that softmax is applied to FP32 tensors* (since softmax in FP16 mode loses too much precision).

Question

Please report your final validation accuracy after 2 epochs in your written assignment, along with screenshots of the training loss and the validation loss displayed on Tensorboard.

```
! pip install accelerate -U
```

```
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.12.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from accelerate) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from accelerate) (25.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from accelerate) (6.0.3)
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from accelerate) (2.9.0+cu126)
Requirement already satisfied: huggingface_hub>=0.21.0 in /usr/local/lib/python3.12/dist-packages (from accelerate) (0.7.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from accelerate) (0.7.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (3.12.2)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (2023.12.1)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (2.31.0)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (4.66.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (4.10.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (1.1.5)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (68.0.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.3)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.1.2)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.3.20)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (1.11.1.6)
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.5.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.0.0->accelerate) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=2.0.0->accelerate) (2.1.5)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface_hub) (3.3.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface_hub) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface_hub) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface_hub) (2025.1.1)
```

```
import os
os.environ["WANDB_DISABLED"] = "true"
```

```
training_args = Seq2SeqTrainingArguments(
    output_dir="my_xsum_model",
```

```
    learning_rate=1e-4,
    weight_decay=0.01,
    warmup_ratio=0.1,
```

```
    per_device_train_batch_size=16,
    gradient_accumulation_steps=1,
    dataloader_drop_last=True,
    per_device_eval_batch_size=16,
    num_train_epochs=2
```

```

num_train_epochs=2,

predict_with_generate=False,
push_to_hub=False,
logging_dir="my_xsum_model/logs",
logging_strategy="steps",
logging_first_step=True,
logging_steps=1,
save_strategy="epoch",
eval_strategy="epoch",
fp16=True,
save_safetensors=False,
)

trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset_train,
    eval_dataset=dataset_validation,
    tokenizer=tokenizer,
    data_collator=data_collator,
)

trainer.train()

```

Using the `WANDB\_DISABLED` environment variable is deprecated and will be removed in v5. Use the `--report_to` flag to `/tmp/ipython-input-1816706699.py:29: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0 for`

```

trainer = Seq2SeqTrainer(

```

 [3042/3042 20:10, Epoch 2/2]

Epoch	Training Loss	Validation Loss
1	3.998600	4.472611
2	4.017400	4.357763

```

TrainOutput(global_step=3042, training_loss=4.1757695839485, metrics={'train_runtime': 1211.1566,
'train_samples_per_second': 40.209, 'train_steps_per_second': 2.512, 'total_flos': 0.0, 'train_loss':
4.1757695839485, 'epoch': 2.0})

```

Notes: We see that the validation loss is below 5 for both epochs showing us that the model architecture that Cursor built successfully converged as intended. This whole process was done zero-shot which is even more impressive in my eyes.

✓ Submission

You are done with this coding assignment. Please download `submission_log.json` and submit it to Gradescope.

You might be thinking: wait, why hasn't the model been tested on real evaluation metrics like ROUGE score? Why haven't we looked at any examples of the model's generated text yet? That's because this is Part I of the assignment.

The Transformer model you implemented above cannot be used for efficient autoregressive generation. For each token that needs to be decoded, it needs to run a forward pass of the encoder for the document and a forward pass of the decoder for all tokens in the generated summary. This would be disastrously slow even for a single example.

In the following weeks, you will work on Part II, which will cover efficient sequence generation using a Transformer by caching computed encoder/decoder states. Moreover, you will learn to fine-tune pretrained language models for significantly better performance.

```

with open("submission_logs.json", "w", encoding="utf-8") as f:
    json.dump(TO_SAVE, f)

```


Putting Example Difficulty in Practice with Early Exit

In the last notebook we explored the concept of example difficulty and prediction depth. How can we use this knowledge to our advantage? What kind of benefits can we see by leveraging our understanding of example difficulty.

In many growing domains of machine learning, we often will have systems constraints like inference latency, energy usage, etc.

As we noticed in the last homework, some examples actually don't need to be passed through the entire network. This concept was formalized as prediction depth.

So why do we need to pass the inputs through the entire network if they can be predicted correctly by an earlier layer?

Are there tradeoffs associated with not going through the entire network?

Much of this homework was inspired by the following paper:

<https://arxiv.org/abs/1709.01686>

Concepts of BranchyNet

We will now have N exits, as we did with the KNN classifiers. However, now each exit will contribute to the loss in the following manner

$$L_{\text{early exit}}(\hat{y}, y; \theta) = \sum_{i=1}^N w_i L(y_{\text{exit}}, y; \theta)$$

We will set the total loss of the network be a weighted sum of the standard cross entropy losses at each exit

Baseline ResNet-18

To properly see the effects of Early Exit, let's set up a resnet without early exit as a baseline for computation and inference speed.

```
In [1]: import torch
import torch.nn as nn
import copy
import torchvision
import torch.optim as optim
import torchvision.transforms as transforms
```

```
import sklearn
from sklearn.neighbors import KNeighborsClassifier
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import entropy
from tqdm import tqdm
from architectures import EarlyExitResNet18
from copy import deepcopy
```

```
In [2]: batch_size = 256
```

MACS

MACS stands for multiply and accumulate - In the hardware, this corresponds to multiplying, then adding a number to an accumulator. We use MACS as a measurement of the amount of computation used.

```
In [3]: !pip install torchprofile
```

Collecting torchprofile

Downloading torchprofile-0.0.4-py3-none-any.whl.metadata (303 bytes)
Requirement already satisfied: numpy>=1.14 in /usr/local/lib/python3.12/dist-packages (from torchprofile) (2.0.2)
Requirement already satisfied: torch>=1.4 in /usr/local/lib/python3.12/dist-packages (from torchprofile) (2.9.0+cu126)
Requirement already satisfied: torchvision>=0.4 in /usr/local/lib/python3.12/dist-packages (from torchprofile) (0.24.0+cu126)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (3.6)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (12.5.4.2)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (12.5.4.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (3.3.20)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (1.11.1.6)
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.4->torchprofile) (3.5.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.4->torchprofile) (11.3.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.4->torchprofile) (1.3.0)
 Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.4->torchprofile) (3.0.3)
 Downloading torchprofile-0.0.4-py3-none-any.whl (7.7 kB)
 Installing collected packages: torchprofile
 Successfully installed torchprofile-0.0.4

```
In [4]: import torchprofile

def get_model_macs(model, inputs) -> int:
    return torchprofile.profile_macs(model, inputs)
```

Let's set up our dataloader in the standard fashion. Similar to last homework, please download the data and put it in the same folder as this

```
In [5]: data = np.load('data.npy', allow_pickle=True).item()
x_tensor = torch.FloatTensor(data['x'])
y_tensor = torch.LongTensor(data['y'])
dataset = torch.utils.data.TensorDataset(x_tensor, y_tensor)

test_data = np.load('test_data.npy', allow_pickle=True).item()
x_tensor = torch.FloatTensor(test_data['x'])
y_tensor = torch.LongTensor(test_data['y'])
test_dataset = torch.utils.data.TensorDataset(x_tensor, y_tensor)

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

transform = transforms.Compose([
    transforms.ToPILImage(),                # Convert arrays to PIL images
    transforms.Grayscale(num_output_channels=3), # Convert grayscale to RGB
    transforms.Resize((224, 224)),          # Resize all images to 224x224
    transforms.ToTensor(),                  # Convert the images to PyTorch tensors
])

resnet_dataset = deepcopy(dataset)
resnet_test_dataset = deepcopy(test_dataset)

resnet_dataset.transform = transform
resnet_trainloader = torch.utils.data.DataLoader(resnet_dataset, batch_size=32)

resnet_test_dataset.transform = transform
resnet_testloader = torch.utils.data.DataLoader(resnet_test_dataset, batch_size=32)
```

```
In [6]: resnet = EarlyExitResNet18()
resnet = resnet.to(device)
```

```
In [7]: criterion = nn.CrossEntropyLoss()
resnet_optimizer = optim.Adam(resnet.parameters(), lr=0.0001)
```

Training

Let's train a standard ResNet. Don't forget to pass in an entropy tolerance and set `early_exit` to `False`

```
In [8]: step = 0
resnet_losses = []
for epoch in tqdm(range(10)): # loop over the dataset multiple times
    running_loss = 0.0
    for i, data in enumerate(resnet_trainloader, 0):
        step += 1
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = inputs, labels = data[0].to(device), data[1].to(device)

        inputs = inputs.unsqueeze(1)
        inputs = inputs.repeat(1, 3, 1, 1)
        inputs = inputs.to(device)

        # zero the parameter gradients
        resnet_optimizer.zero_grad()

        # forward + backward + optimize
        # TODO
        outputs = resnet(inputs, 0, False)
        loss = criterion(outputs, labels)
        loss.backward()
        resnet_optimizer.step()
        resnet_losses.append(loss.item())
        # print statistics
        running_loss += loss.item()
        if i % 50 == 49: # print every 2000 mini-batches
            print(f'[epoch + 1], {i + 1}] loss: {running_loss / 50:.3f}')
            running_loss = 0.0

print('Finished Training')
```

```
0%|          | 0/10 [00:00<?, ?it/s]
[1, 50] loss: 1.156
[1, 100] loss: 0.405

10%|█         | 1/10 [00:29<04:27, 29.68s/it]
[2, 50] loss: 0.146
[2, 100] loss: 0.136

20%|██        | 2/10 [00:57<03:50, 28.80s/it]
[3, 50] loss: 0.067
[3, 100] loss: 0.072

30%|███       | 3/10 [01:27<03:23, 29.06s/it]
[4, 50] loss: 0.031
[4, 100] loss: 0.038

40%|████      | 4/10 [01:56<02:55, 29.17s/it]
[5, 50] loss: 0.022
[5, 100] loss: 0.019

50%|█████     | 5/10 [02:25<02:25, 29.01s/it]
```

```

[6, 50] loss: 0.013
[6, 100] loss: 0.015
60%|██████    | 6/10 [02:54<01:56, 29.05s/it]
[7, 50] loss: 0.017
[7, 100] loss: 0.018
70%|██████    | 7/10 [03:23<01:27, 29.08s/it]
[8, 50] loss: 0.015
[8, 100] loss: 0.011
80%|██████    | 8/10 [03:52<00:58, 29.06s/it]
[9, 50] loss: 0.008
[9, 100] loss: 0.009
90%|██████    | 9/10 [04:21<00:29, 29.06s/it]
[10, 50] loss: 0.005
[10, 100] loss: 0.004
100%|██████████| 10/10 [04:50<00:00, 29.08s/it]
Finished Training

```

Notes: This code is very standard for torch training scripts, so it isn't as suprising that Cursor is able to complete it, but still very interesting nonetheless.

Fill in the code below to evaluate the ResNet

```

In [9]: resnet.eval()
total_macs = 0

for epoch in range(1): # loop over the dataset multiple times

    total_correct = 0
    with torch.no_grad():
        for i, data in tqdm(enumerate(resnet_testloader, 0)):
            # get the inputs; data is a list of [inputs, labels]

            inputs, labels = inputs, labels = data[0].to(device), data[1].to(device)

            inputs = inputs.unsqueeze(1)
            inputs = inputs.repeat(1, 3, 1, 1)
            inputs = inputs.to(device)
            # forward + backward + optimize
            # TODO
            outputs = resnet(inputs, 0, False)
            total_macs += get_model_macs(resnet, (inputs, 0, False))

            indices = torch.argmax(outputs, dim=1)

            total_correct += torch.sum(labels == indices)

print(f'Accuracy: {total_correct/6000} %')
print('Total MACS: ', total_macs)

47it [00:04, 9.54it/s]

```

Accuracy: 0.903499960899353 %
 Total MACS: 3336213504000

Notes: With the basic implementation, we see an accuracy of 90% which is expected given the previous code snippets.

What was the accuracy with a regular ResNet-18?

Inference Speed?

Total MACS?

```
In [10]: resnet_early = EarlyExitResNet18()
resnet_early = resnet_early.to(device)
criterion = nn.CrossEntropyLoss()
resnet_optimizer = optim.Adam(resnet_early.parameters(), lr=0.0001)
```

Fill in the code below to train the early exit network

```
In [11]: step = 0
resnet_losses = []
for epoch in tqdm(range(10)): # loop over the dataset multiple times
    running_loss = 0.0
    for i, data in enumerate(resnet_trainloader, 0):
        step += 1
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = inputs, labels = data[0].to(device), data[1].to(device)

        inputs = inputs.unsqueeze(1)
        inputs = inputs.repeat(1, 3, 1, 1)
        inputs = inputs.to(device)

        # zero the parameter gradients
        resnet_optimizer.zero_grad()

        # forward + backward + optimize
        # TODO Use w_0 = 1 and w_i = 0.3 for i > 0
        outputs = resnet_early.early_train(inputs)
        loss = 1.0 * criterion(outputs[3], labels) + 0.3 * (criterion(outputs[0], labels) + criterion(outputs[1], labels) + criterion(outputs[2], labels))
        loss.backward()
        resnet_optimizer.step()
        resnet_losses.append(loss.item())
        # print statistics
        running_loss += loss.item()
        if i % 50 == 49: # print every 2000 mini-batches
            print(f'[epoch + 1], {i + 1}] loss: {running_loss / 50:.3f}')
            running_loss = 0.0

    print('Finished Training')
```

```

0%|          | 0/10 [00:00<?, ?it/s]
[1, 50] loss: 2.575
[1, 100] loss: 1.258
10%|█        | 1/10 [00:30<04:32, 30.27s/it]
[2, 50] loss: 0.632
[2, 100] loss: 0.537
20%|██       | 2/10 [01:00<04:00, 30.01s/it]
[3, 50] loss: 0.360
[3, 100] loss: 0.370
30%|███      | 3/10 [01:29<03:27, 29.68s/it]
[4, 50] loss: 0.261
[4, 100] loss: 0.247
40%|████     | 4/10 [01:59<02:58, 29.71s/it]
[5, 50] loss: 0.188
[5, 100] loss: 0.201
50%|█████    | 5/10 [02:28<02:28, 29.68s/it]
[6, 50] loss: 0.133
[6, 100] loss: 0.116
60%|██████   | 6/10 [02:58<01:58, 29.64s/it]
[7, 50] loss: 0.092
[7, 100] loss: 0.099
70%|███████  | 7/10 [03:27<01:28, 29.64s/it]
[8, 50] loss: 0.073
[8, 100] loss: 0.067
80%|████████ | 8/10 [03:57<00:59, 29.65s/it]
[9, 50] loss: 0.051
[9, 100] loss: 0.053
90%|█████████| 9/10 [04:27<00:29, 29.61s/it]
[10, 50] loss: 0.044
[10, 100] loss: 0.057
100%|██████████| 10/10 [04:56<00:00, 29.68s/it]
Finished Training

```

Notes: We see a successful zero-shot approach to solving the arly exit training loop, which mirrors common implementations with torch.

Fill in the code below to evaluate the early exit network

```

In [12]: resnet_early.eval()
         exiting_points = {0:0, 1:0, 2:0, 3:0}
         entropies = []
         total_early_macs = 0

         for epoch in range(1): # loop over the dataset multiple times

             total_early_correct = 0
             with torch.no_grad():
                 for i, data in tqdm(enumerate(resnet_testloader, 0)):
                     # get the inputs; data is a list of [inputs, labels]

                     inputs, labels = inputs, labels = data[0].to(device), data[1].to

```

```

inputs = inputs.unsqueeze(1)
inputs = inputs.repeat(1, 3, 1, 1)
inputs = inputs.to(device)
# forward + backward + optimize
# TODO Use an entropy tolerance of 0.05
outputs, num, curr_entropy = resnet_early(inputs, 0.05, True)
entropies.append(curr_entropy)
total_early_macs += get_model_macs(resnet_early, (inputs, 0.05))

exiting_points[num] += 1

indices = torch.argmax(outputs, dim=1)

total_early_correct += torch.sum(labels == indices)

print(f'Accuracy: {total_early_correct/6000} %')
print('Num Exiting: ', exiting_points)
print('Total MACS: ', total_early_macs)
entropies = sorted(entropies)
print(len(entropies))
print('Entropies: ', entropies[0], entropies[len(entropies)//4], entropies[1

```

47it [00:05, 8.31it/s]

Accuracy: 0.9424999952316284 %

Num Exiting: {0: 0, 1: 13, 2: 6, 3: 28}

Total MACS: 2786615697408

47

Entropies: 0.0044781445 0.02942098 0.04835912 0.10422034 0.23158494

Notes: The model is able to achieve very high accuracy here above 0.94, which is very impressive for a zero-shot approach for solving this notebook. This demonstrates the importance of early exiting compared to the results from earlier in the notebook. This shows that cursor was able to successfully demonstrate these values while solving problems in the notebook with a zero-shot approach.

What was the accuracy with an early exit ResNet-18?

Inference Speed?

Total MACS?

In [13]: `print(f'Ratio of Standard to Early Exit MACS: {total_macs/total_early_macs}')`

Ratio of Standard to Early Exit MACS: 1.197227700648932

Entropy

Play around with the entropy tolerance to see how low you can get the MACS while keeping 90 percent or greater accuracy.

How did early exit do? Compare accuracy and MACS.

Open Question

No solutions will be provided for this question:

When would we use early exit, versus just using a smaller model? What factors should we consider?

How does early exit relate to example difficulty?

Hand transformer notebook discussion

Exported on 12/4/2025 at 16:33:08 PST from Cursor (2.1.46)

User

Start by implementing “a Simple Transformer” described in the notebook below.
Focus on just the first section labelled “Implement a Simple Transformer”

@hand_transformer/q_hand_transformer.ipynb

Cursor

```
def __init__(self, input_dim, qk_dim, v_dim, pos_dim=None, max_seq_len=10):
    super().__init__()
    if pos_dim is not None:
        self.pos = nn.Embedding(max_seq_len, pos_dim)
    else:
        self.pos = None
    in_dim = input_dim

    if pos_dim is not None:
        in_dim += pos_dim

    # Define query, key, value projection layers
    self.Km = nn.Linear(in_dim, qk_dim, bias=False)
    self.Qm = nn.Linear(in_dim, qk_dim, bias=False)
    self.Vm = nn.Linear(in_dim, v_dim, bias=False)

    self.d_k = qk_dim

def forward(self, seq):
    # seq is (seq_len, input_dim)

    # Concatenate positional encodings if they exist
    if self.pos is
```

I'll continue the implementation by completing the partially written forward method. I'll use

The positional encoding logic requires adding embeddings if they exist. I'll carefully implement

When positional embeddings are present, I generate position indices, compute embeddings, and

The key steps involve:

1. Optional positional embedding concatenation

2. Projecting sequence into Q, K, V spaces
3. Computing attention scores
4. Applying softmax normalization
5. Generating output via weighted value aggregation

Looking at the notebook, I need to implement the `PytorchTransformer` class to match the behavior of the `NumpyTransformer`. Let me analyze the `NumpyTransformer` first to understand what needs to be implemented:

1. `__init__`: Define `Km`, `Qm`, `Vm` as linear projections without bias
2. `forward`:
 - Concatenate positional encodings if they exist
 - Compute K, Q, V projections
 - Apply scaled dot-product attention: $\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$

Let me implement this:

```
class PytorchTransformer(nn.Module):
    def __init__(self, input_dim, qk_dim, v_dim, pos_dim=None, max_seq_len=10):
        super().__init__()
        if pos_dim is not None:
            self.pos = nn.Embedding(max_seq_len, pos_dim)
        else:
            self.pos = None
        in_dim = input_dim
        if pos_dim is not None:
            in_dim += pos_dim

        #####
        # TODO: Define query, key, value projection layers Qm, Km, Vm.
        #       Each of them is a linear projection without bias
        #####

        self.d_k = qk_dim

    def forward(self, seq):
        """
        Transformer forward pass

        Inputs: seq is a torch tensor of shape (seq_len, input_dim).
        Outputs: a torch tensor of shape (seq_len, v_dim), the output of the attention operation.
        """
        #####
        # TODO: Implement the forward pass of the `PytorchTransformer` class.
        #       The forward pass should be identical to the forward pass of the
        #       `NumpyTransformer` class.
        #
```

```

# Hint: The attention operation should be implemented as
#       If `pos` exists, it should be concatenated to the input sequence.
#####
#####
# END OF YOUR CODE
#####
return out

test()

->

class PytorchTransformer(nn.Module):
    def __init__(self, input_dim, qk_dim, v_dim, pos_dim=None, max_seq_len=10):
        super().__init__()
        if pos_dim is not None:
            self.pos = nn.Embedding(max_seq_len, pos_dim)
        else:
            self.pos = None
        in_dim = input_dim
        if pos_dim is not None:
            in_dim += pos_dim

        #####
        # TODO: Define query, key, value projection layers Qm, Km, Vm.
        #       Each of them is a linear projection without bias
        #####
        self.Km = nn.Linear(in_dim, qk_dim, bias=False)
        self.Qm = nn.Linear(in_dim, qk_dim, bias=False)
        self.Vm = nn.Linear(in_dim, v_dim, bias=False)
        #####

        self.d_k = qk_dim

    def forward(self, seq):
        """
        Transformer forward pass

        Inputs: seq is a torch tensor of shape (seq_len, input_dim).
        Outputs: a torch tensor of shape (seq_len, v_dim), the output of the attention operation
        """
        #####
        # TODO: Implement the forward pass of the `PytorchTransformer` class.
        #       The forward pass should be identical to the forward pass of the
        #       `NumpyTransformer` class.
        #

```

```

# Hint: The attention operation should be implemented as
#       If `pos` exists, it should be concatenated to the input sequence.
#####
# Concatenate positional encodings if they exist
if self.pos is not None:
    seq_len = seq.shape[0]
    positions = torch.arange(seq_len)
    pos_emb = self.pos(positions) # (seq_len, pos_dim)
    seq = torch.cat([seq, pos_emb], dim=-1) # (seq_len, input_dim + pos_dim)

# Compute K, Q, V projections
K = self.Km(seq) # (seq_len, qk_dim)
Q = self.Qm(seq) # (seq_len, qk_dim)
V = self.Vm(seq) # (seq_len, v_dim)

# Compute attention scores:  $Q @ K^T / \sqrt{d_k}$ 
scores = Q @ K.T / math.sqrt(self.d_k) # (seq_len, seq_len)

# Apply softmax to get attention weights (along last dim, i.e., over keys)
attn_weights = torch.softmax(scores, dim=-1) # (seq_len, seq_len)

# Compute output: attention_weights @ V
out = attn_weights @ V # (seq_len, v_dim)
#####
# END OF YOUR CODE
#####
return out

```

test()

Let me read the specific cell to see the exact contents:

I see the issue - the notebook stores cells in JSON format with different escaping.

Let me try editing by running a grep to find the exact cell index and structure:

Let me check the structure more carefully to find the right cell index:

The PytorchTransformer class is in cell index 6. Let me edit it:

```

#####
# TODO: Define query, key, value projection layers Qm, Km, Vm.
#       Each of them is a linear projection without bias
#####
#####

self.d_k = qk_dim

```

->

```
#####
# TODO: Define query, key, value projection layers Qm, Km, Vm.
#       Each of them is a linear projection without bias
#####
self.Km = nn.Linear(in_dim, qk_dim, bias=False)
self.Qm = nn.Linear(in_dim, qk_dim, bias=False)
self.Vm = nn.Linear(in_dim, v_dim, bias=False)
#####

self.d_k = qk_dim
```

Now let me implement the forward pass:

```
#####
# TODO: Implement the forward pass of the `PytorchTransformer` class.
#       The forward pass should be identical to the forward pass of the
#       `NumpyTransformer` class.
#
# Hint: The attention operation should be implemented as
#       If `pos` exists, it should be concatenated to the input sequence.
#####
# END OF YOUR CODE
#####
return out
```

->

```
#####
# TODO: Implement the forward pass of the `PytorchTransformer` class.
#       The forward pass should be identical to the forward pass of the
#       `NumpyTransformer` class.
#
# Hint: The attention operation should be implemented as
#       If `pos` exists, it should be concatenated to the input sequence.
#####
# Concatenate positional encodings if they exist
if self.pos is not None:
    seq_len = seq.shape[0]
    positions = torch.arange(seq_len)
    pos_emb = self.pos(positions) # (seq_len, pos_dim)
    seq = torch.cat([seq, pos_emb], dim=-1) # (seq_len, input_dim + pos_dim)

# Compute K, Q, V projections
K = self.Km(seq) # (seq_len, qk_dim)
Q = self.Qm(seq) # (seq_len, qk_dim)
```

```

V = self.Vm(seq) # (seq_len, v_dim)

# Compute attention scores: Q @ K^T / sqrt(d_k)
scores = Q @ K.T / math.sqrt(self.d_k) # (seq_len, seq_len)

# Apply softmax to get attention weights (along last dim, i.e., over keys)
attn_weights = torch.softmax(scores, dim=-1) # (seq_len, seq_len)

# Compute output: attention_weights @ V
out = attn_weights @ V # (seq_len, v_dim)
#####
# END OF YOUR CODE
#####
return out

```

I've implemented the `PytorchTransformer` class. Let me run the test to verify it works:

Let me use the conda environment that the notebook uses:

User

Now, continue by implementing “self-attention: Attention by Content”. we will explore how Transformers can attend to different tokens in a variable-length sequence based on their contents. We will do this by **implementing a Transformer that performs the *identity* operation on a sequence of one-hot vectors**. We will then compare the performance and weights of this hand-coded Transformer with those of a PyTorch model trained on the same task. Refer to the corresponding section in the notebook for more information.

Cursor

Looking at the “Self-Attention: Attention by Content” section, I need to implement K_m , Q_m , and V_m matrices that make the transformer perform the identity operation on one-hot vectors.

The key insight is: - For one-hot vectors, if we use the identity matrix as our projection, the keys and queries will be the one-hot vectors themselves - The dot product between matching tokens will be 1, and between non-matching tokens will be 0 - We need to scale up the keys/queries to make the softmax sharply attend to matching tokens - Values should just preserve the original one-hot encoding

Let me find the exact cell to edit:

Now let me read cell 9 to see the exact content:

Let me find the correct cell for “Attention by Content”:

Now I'll implement the K_m , Q_m , and V_m matrices for the identity operation. The strategy is: 1. Use scaled identity matrices for K_m and Q_m so matching one-hot vectors have high dot products 2. Use identity for V_m to preserve the original values

```
#####
# TODO: Write Numpy arrays for `Km`, `Qm`, and `Vm`.
#       The dimensions should be (input_dim, qk_dim), (input_dim, qk_dim), and
#       (input_dim, v_dim), respectively.
#       In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
#       choose, but 3 is a reasonable choice.
#####
##### END OF YOUR CODE #####

->

#####
# TODO: Write Numpy arrays for `Km`, `Qm`, and `Vm`.
#       The dimensions should be (input_dim, qk_dim), (input_dim, qk_dim), and
#       (input_dim, v_dim), respectively.
#       In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
#       choose, but 3 is a reasonable choice.
#####
# For identity operation: each token should attend to tokens with the same content
# Using scaled identity matrices:
# - Km and Qm: scale up to make attention sharp (high scores for matching tokens)
# - Vm: identity to preserve original values
scale = 10
Km = np.eye(3) * scale # (3, 3) - Keys encode token identity
Qm = np.eye(3) * scale # (3, 3) - Queries look for matching tokens
Vm = np.eye(3)         # (3, 3) - Values are the original tokens
##### END OF YOUR CODE #####
```

Now let me test the implementation:

The implementation works correctly. Let me explain the solution:

Solution for “Attention by Content” (Identity Operation)

Key insight: For one-hot vectors, the identity operation requires each token to attend only to tokens with the **same content**.

Design choices: - $K_m = Q_m = 10 \times I$ (scaled identity): - For matching tokens (same one-hot encoding): dot product = $10^2 = 100$ - For non-matching tokens: dot product = 0 - After scaling by $\sqrt{d_k}$ and softmax, attention concentrates on matching tokens

- $V_m = I$ (identity):

- Values are the original one-hot vectors
- Weighted sum preserves the original token

Why it works: - Token A (query [10,0,0]) has high attention scores with all A keys (dot=100) and zero with B,C keys (dot=0) - After softmax, attention weights are ~1.0 for matching and ~0 for non-matching - Since repeated tokens (like C,C) have identical values, averaging them still gives C

The tests all pass, showing the output is essentially identical to the input (numerical errors $\sim 10^{-2}$).

User

Now, write the section for “Self-Attention: Attention by Position”. Similar to before, show your reasoning and complete the TODOs in this section

Cursor

Looking at the “Self-Attention: Attention by Position” section, I need to implement a transformer that copies the first token across the entire sequence: - Input: [A, B, C, C] → Output: [A, A, A, A] - Input: [C, A, C] → Output: [C, C, C]

Let me first read the exact cell to understand the requirements:

Let me find the exact cell index:

Now let me implement the solution for “Attention by Position”.

Reasoning

The task is to make **all tokens attend to position 0** (the first token) regardless of their content or position.

Design Strategy: 1. **Positional encodings (pos):** Create encodings where:
 - First dimension: constant 1 for ALL positions → allows queries to be identical
 - Second dimension: 10 only for position 0, 0 for others → makes position 0's key stand out

2. **Keys (Km):** Extract only the “position 0 indicator” (second pos dimension)
 - $\text{Key}[0] = 10$ (high), $\text{Key}[1,2,3] = 0$ (low)
3. **Queries (Qm):** Extract the constant dimension (first pos dimension)
 - $\text{Query}[\text{all positions}] = 1$ (identical queries)
4. **Values (Vm):** Extract only content (first 3 dims), ignore position
 - Output will be the original token content

This way: $Q @ K^T = [1 \times 10, 1 \times 0, 1 \times 0, \dots] \rightarrow \text{softmax concentrates on position 0!}$

```

#####
# TODO: Implement numpy arrays for Km, Qm, and Vm and pos.
#       The shape of Km, and Qm are [input_dim + pos_dim, qk_dim].
#       The shape of Vm is [input_dim + pos_dim, v_dim].
#       The shape of pos is [max_len, pos_dim].
#       In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
#       choose, but 1 is a reasonable choice. max_len is the maximum sequence
#       length you will encounter, 4 in this case.
#       pos_dim can be any value you choose, but 4 is a resonable choice.
#####
##### END OF YOUR CODE #####

->

#####
# TODO: Implement numpy arrays for Km, Qm, and Vm and pos.
#       The shape of Km, and Qm are [input_dim + pos_dim, qk_dim].
#       The shape of Vm is [input_dim + pos_dim, v_dim].
#       The shape of pos is [max_len, pos_dim].
#       In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
#       choose, but 1 is a reasonable choice. max_len is the maximum sequence
#       length you will encounter, 4 in this case.
#       pos_dim can be any value you choose, but 4 is a resonable choice.
#####
# Strategy: All positions should attend to position 0 (the first token)
# - pos: first dim is constant 1 (for uniform queries), second dim marks position 0
# - Km: extracts position 0 indicator → Key[0]=high, Key[others]=0
# - Qm: extracts constant dim → Query[all]=1 (same query for all positions)
# - Vm: extracts only content (ignores position) → output is original token

# Positional encodings: shape (max_len, pos_dim) = (4, 2)
# First dim: constant 1 for all positions (used by queries)
# Second dim: 10 for position 0, 0 for others (used by keys)
pos = np.array([
    [1, 10], # position 0 - has high value in second dim
    [1, 0],  # position 1
    [1, 0],  # position 2
    [1, 0],  # position 3
])

# Input after concatenation: [content (3 dims) | pos (2 dims)] = 5 dims
# Km extracts second pos dim (position 0 indicator): shape (5, 1)
Km = np.array([[0], [0], [0], [0], [1]])

# Qm extracts first pos dim (constant 1 for all): shape (5, 1)
Qm = np.array([[0], [0], [0], [1], [0]])

```



```
# Vm extracts only content (first 3 dims), ignores position: shape (5, 3)
Vm = np.array([
    [1, 0, 0],
    [0, 1, 0],
    [0, 0, 1],
    [0, 0, 0],
    [0, 0, 0]
])
##### END OF YOUR CODE #####
```

Now let me test the implementation:

I see the issue - the attention is at 99.99% but not sharp enough. The outputs like [0.9999, 0.00005, 0.00005] don't match [1, 0, 0] within the tolerance because of small leakage to other positions. I need to increase the scale factor to make attention sharper:

```
# Positional encodings: shape (max_len, pos_dim) = (4, 2)
# First dim: constant 1 for all positions (used by queries)
# Second dim: 10 for position 0, 0 for others (used by keys)
pos = np.array([
    [1, 10], # position 0 - has high value in second dim
    [1, 0],  # position 1
    [1, 0],  # position 2
    [1, 0],  # position 3
])
```

->

```
# Positional encodings: shape (max_len, pos_dim) = (4, 2)
# First dim: constant 1 for all positions (used by queries)
# Second dim: large value (100) for position 0, 0 for others (used by keys)
# Using 100 makes attention very sharp (softmax concentrates almost entirely on pos 0)
pos = np.array([
    [1, 100], # position 0 - has high value in second dim
    [1, 0],   # position 1
    [1, 0],   # position 2
    [1, 0],   # position 3
])
```

All tests pass. Let me verify with a few more specific examples:

All tests pass. Here's a summary of the solution:

Solution for “Attention by Position” (Copy First Token)

Task: Every position should output the content of the first token.

Design Strategy:

Component	Shape	Purpose
pos	(4, 2)	First dim = constant 1 (for queries), Second dim = 100 only at pos 0 (for keys)
Km	(5, 1)	Extracts position 0 indicator → Key[0]=100, Key[others]=0
Qm	(5, 1)	Extracts constant dimension → Query[all]=1
Vm	(5, 3)	Extracts only content (ignores position)

How it works: 1. All queries are identical: $Q = [1, 1, 1, 1]$ 2. Only position 0's key is high: $K = [100, 0, 0, 0]$ 3. $Q @ K^T = [[100, 0, 0, 0], [100, 0, 0, 0], ...]$ for all rows 4. After softmax: attention $[1, 0, 0, 0] \rightarrow$ all positions attend to position 0 5. Output = attention $\times V = V[0]$ = first token's content repeated

User

As a challenge, try completing the “Self-Attention: Attention by Content and Position” section.

Cursor

Input sequence --> Input sequence with CLS and pos encoding --> Output sequence
[A, B, C, C] --> [CLS: pos_0, A:pos_1, B:pos_2, C:pos_3, C:pos_4] --> [Ignore, 1, 1, -1, -1]
[C, A, C] --> [CLS: pos_0, C:pos_1, A:pos_2, C:pos_3] --> [Ignore, -1, 1, -1]
[B, B, C] --> [CLS: pos_0, B:pos_1, B:pos_2, C:pos_3] --> [Ignore, -1, -1, 1]

This is a challenging problem! Let me analyze the task and design a solution.

Task Analysis

For each token in a sequence, output: - **Positive** if the token is **unique** (appears exactly once) - **Negative** if the token is **repeated** (appears more than once)

Examples: - [A, B, C, C] $\rightarrow$ [1, 1, -1, -1] (A and B unique, both Cs repeated) - [C, A, C] $\rightarrow$ [-1, 1, -1] (C repeated twice, A unique)

Design Strategy

The key insight is to use: 1. **Content matching** to find same-content tokens 2. **Position exclusion** to NOT attend to self (same position) 3. **CLS baseline** to provide positive output when unique (no duplicates found)

Let me first understand the cell structure:

Now I understand the structure. Let me implement the solution for this challenging problem.

Design Reasoning

Key Insight: For each token, I want to: 1. Find OTHER tokens with the SAME content (but different position) → indicates duplicates 2. If no duplicates exist, fall back to CLS → indicates unique

Score Design: - Same content + Same position: **excluded** (self-attention blocked) - Same content + Different position: **high score** (found a duplicate!) - CLS token: **moderate score** (baseline when no duplicates)

Value Design: - CLS value: **positive** (unique indicator) - Other token values: **negative** (duplicate indicator)

```
#####
# TODO: Implement numpy arrays for Km, Qm, and Vm and pos.
#     The dimensions of Km, and Qm are (input_dim + pos_dim, qk_dim).
#     The dimensions of Vm are (input_dim + pos_dim, v_dim).
#     The dimensions of pos are (max_len + 1, pos_dim). (Each row is a position vector.)
#     In this case, input_dim = 4, and v_dim = 1. qk_dim can be any value you choose, but 8
#     a reasonable choice. max_len is the maximum sequence length you will encounter (before
#     4 in this case. pos_dim can be any value you choose, but 4 is a reasonable choice.
#####
##### END OF YOUR CODE #####
```

->

```
#####
# TODO: Implement numpy arrays for Km, Qm, and Vm and pos.
#     The dimensions of Km, and Qm are (input_dim + pos_dim, qk_dim).
#     The dimensions of Vm are (input_dim + pos_dim, v_dim).
#     The dimensions of pos are (max_len + 1, pos_dim). (Each row is a position vector.)
#     In this case, input_dim = 4, and v_dim = 1. qk_dim can be any value you choose, but 8
#     a reasonable choice. max_len is the maximum sequence length you will encounter (before
#     4 in this case. pos_dim can be any value you choose, but 4 is a reasonable choice.
#####
# Strategy:
```

```

# - For each query, attend to same-content tokens at DIFFERENT positions (to find duplicates)
# - If duplicates exist → attend to them → negative value → negative output
# - If unique → attend to CLS (fallback) → positive value → positive output
#
# Attention scores:
# - Same content + same position: EXCLUDED (content matches, position cancels)
# - Same content + different position: HIGH (found duplicate!)
# - CLS: moderate (baseline when no duplicates)

# Positional encoding (5 x 5): [CLS_indicator, pos1, pos2, pos3, pos4]
# - First dim: high for CLS (10), small for others (1) → allows queries to find CLS
# - Dims 1-4: position one-hot scaled (10) → for self-exclusion
pos = np.array([
    [10, 0, 0, 0, 0], # CLS (position 0): high in CLS indicator dim
    [1, 10, 0, 0, 0], # position 1
    [1, 0, 10, 0, 0], # position 2
    [1, 0, 0, 10, 0], # position 3
    [1, 0, 0, 0, 10], # position 4
])

# Input structure: [content (4 dims: A,B,C,CLS) | pos (5 dims)] = 9 dims
# qk_dim = 9: [content_match (4) | pos_exclusion (4) | CLS_indicator (1)]

scale = 10

# Keys (Km): 9 x 9
# - content → content (scaled, for matching)
# - pos dims 1-4 → exclusion dims (scaled, for being excluded)
# - pos dim 0 → CLS indicator (for CLS attention)
Km = np.array([
    [scale, 0, 0, 0, 0, 0, 0, 0, 0], # content A
    [0, scale, 0, 0, 0, 0, 0, 0, 0], # content B
    [0, 0, scale, 0, 0, 0, 0, 0, 0], # content C
    [0, 0, 0, scale, 0, 0, 0, 0, 0], # content CLS
    [0, 0, 0, 0, 0, 0, 0, 0, 1], # pos dim 0 (CLS indicator) → output dim 8
    [0, 0, 0, 0, scale, 0, 0, 0, 0], # pos dim 1 → exclusion dim 4
    [0, 0, 0, 0, 0, scale, 0, 0, 0], # pos dim 2 → exclusion dim 5
    [0, 0, 0, 0, 0, 0, scale, 0, 0], # pos dim 3 → exclusion dim 6
    [0, 0, 0, 0, 0, 0, 0, scale, 0], # pos dim 4 → exclusion dim 7
])

# Queries (Qm): 9 x 9
# - content → content (scaled, for matching)
# - pos dims 1-4 → exclusion dims (NEGATIVE scaled, to penalize same position)
# - pos dim 0 → CLS indicator (to find CLS)
Qm = np.array([

```

```

[0, 0, 0, 0, 0, 0, 0, 0, 0], # content A
[0, 0, 0, 0, 0, 0, 0, 0, 0], # content B
[0, 0, 0, 0, 0, 0, 0, 0, 0], # content C
[0, 0, 0, 0, 0, 0, 0, 0, 0], # content CLS
[0, 0, 0, 0, 0, 0, 0, 0, 1], # pos dim 0 (CLS indicator) → output dim 8
[0, 0, 0, 0, 0, 0, 0, 0, 0], # pos dim 1 → NEGATIVE exclusion
[0, 0, 0, 0, 0, 0, 0, 0, 0], # pos dim 2 → NEGATIVE exclusion
[0, 0, 0, 0, 0, 0, 0, 0, 0], # pos dim 3 → NEGATIVE exclusion
[0, 0, 0, 0, 0, 0, 0, 0, 0], # pos dim 4 → NEGATIVE exclusion
])

```

```

# Values (Vm): 9 x 1
# - CLS content → positive output
# - A, B, C content → negative output
Vm = np.array([
    [-1], # content A → -1
    [-1], # content B → -1
    [-1], # content C → -1
    [5], # content CLS → +5 (positive!)
    [0], # pos dim 0
    [0], # pos dim 1
    [0], # pos dim 2
    [0], # pos dim 3
    [0], # pos dim 4
])

```

END OF YOUR CODE

Now let me test the implementation:

All test cases pass. Let me run a more comprehensive test:

All 20 random tests pass! The implementation is complete.

Summary: Self-Attention by Content and Position

Task: Output positive for unique tokens, negative for repeated tokens.

Design

Component	Purpose
pos (5×5)	Position 0 (CLS) has high value in dim 0; others have small constant in dim 0 + position indicator in dims 1-4
Km (9×9)	Extract content (scaled) + position (scaled) + CLS indicator

Component	Purpose
Q_m (9×9)	Extract content (scaled) + negative position (for exclusion) + CLS indicator
V_m (9×1)	CLS $\rightarrow +5$ (positive), A/B/C $\rightarrow$ -1 (negative)

How It Works

Attention Scores ($Q @ K^T$): - Same content + same position: $100 - 100 + 1 = 1$ (self-excluded!) - Same content + different position: $100 + 0 + 1 = 101$ (duplicate found!)
- CLS: $0 + 0 + 10 = 10$ (baseline) - Different content: low scores

For repeated token: Attention concentrates on same-content-different-position tokens $\rightarrow$ Value -1 $\rightarrow$ **Negative output**

For unique token: No duplicates exist, so attention falls back to CLS $\rightarrow$ Value +5 $\rightarrow$ **Positive output**

Example difficulty and early exit

Exported on 12/7/2025 at 18:46:31 PST from Cursor (2.1.46)

User

Deep Learning Practioners have recognized that within the same task, particular examples in the test set

can actually be harder to perform predictions on than others. Why is that? What kinds of things are easier

to learn? We explore the notion of example difficulty, proposed by Baldock et. al. that will allow us to

perform deeper investigations on the topic. Furthermore, utilizing concepts from example difficulty can aid in development of techniques that can boost inference speeds, sample efficiency, and other key properties that allow scalability in both the micro and macro levels. Here, we are looking into the use cases and effects of example difficulty and early exit.

Fill in the TODOs for the early_exit.ipynb file. @early_exit/early_exit.ipynb @early_exit/example_difficulty.ipynb

Cursor

Reviewing `architectures.py` to understand the `EarlyExitResNet18` structure, then filling in the TODOs.

Filling in the TODOs. Checking the notebook structure:

Reading the specific cells with TODOs:

Filling in the TODOs. Fixing a bug on line 814 where it uses `total_correct` instead of `total_early_correct`.

```
# forward + backward + optimize
# TODO
outputs = ...
loss = ...
loss.backward()
```

->

```
# forward + backward + optimize
# TODO
outputs = resnet(inputs, 0, False)
loss = criterion(outputs, labels)
loss.backward()
```

```

# forward + backward + optimize
# TODO
outputs = ...
total_macs += get_model_macs(resnet, (inputs, 0, False))

->

# forward + backward + optimize
# TODO
outputs = resnet(inputs, 0, False)
total_macs += get_model_macs(resnet, (inputs, 0, False))

# forward + backward + optimize
# TODO Use w_0 = 1 and w_i = 0.3 for i > 0
outputs = ...
loss = ...
loss.backward()

->

# forward + backward + optimize
# TODO Use w_0 = 1 and w_i = 0.3 for i > 0
outputs = resnet_early.early_train(inputs)
loss = 1.0 * criterion(outputs[3], labels) + 0.3 * (criterion(outputs[0], labels) +
loss.backward()

# forward + backward + optimize
# TODO Use an entropy tolerance of 0.05
outputs, num, curr_entropy = ...
entropies.append(curr_entropy)
total_early_macs += get_model_macs(resnet_early, (inputs, 0.05))

->

# forward + backward + optimize
# TODO Use an entropy tolerance of 0.05
outputs, num, curr_entropy = resnet_early(inputs, 0.05, True)
entropies.append(curr_entropy)
total_early_macs += get_model_macs(resnet_early, (inputs, 0.05))

print(f'Accuracy: {total_correct/6000} %')

->

print(f'Accuracy: {total_early_correct/6000} %')

Reading the exact cell contents:

```


Reading the full notebook to understand its structure:

Filling in the TODOs:

```
# forward + backward + optimize
# TODO
outputs = ...
loss = ...

->

# forward + backward + optimize
# TODO
outputs = resnet(inputs, 0, False)
loss = criterion(outputs, labels)

# forward + backward + optimize
# TODO
outputs = ...

->

# forward + backward + optimize
# TODO
outputs = resnet(inputs, 0, False)

# forward + backward + optimize
# TODO Use w_0 = 1 and w_i = 0.3 for i > 0
outputs = ...
loss = ...

->

# forward + backward + optimize
# TODO Use w_0 = 1 and w_i = 0.3 for i > 0
outputs = resnet_early.early_train(inputs)
loss = 1.0 * criterion(outputs[3], labels) + 0.3 * (criterion(outputs[0], labels) +

# forward + backward + optimize
# TODO Use an entropy tolerance of 0.05
outputs, num, curr_entropy = ...

->

# forward + backward + optimize
# TODO Use an entropy tolerance of 0.05
outputs, num, curr_entropy = resnet_early(inputs, 0.05, True)

print(f'Accuracy: {total_correct/6000} %')
```

->

```
print(f'Accuracy: {total_early_correct/6000} %')
```

Checking for another bug in Cell 16:

Fixing a bug in Cell 16 where it uses `resnet` instead of `resnet_early`:

```
resnet_early = EarlyExitResNet18()
resnet_early = resnet.to(device)
criterion = nn.CrossEntropyLoss()
resnet_optimizer = optim.Adam(resnet.parameters(), lr=0.0001)
```

->

```
resnet_early = EarlyExitResNet18()
resnet_early = resnet_early.to(device)
criterion = nn.CrossEntropyLoss()
resnet_optimizer = optim.Adam(resnet_early.parameters(), lr=0.0001)
```

Filled in all TODOs in the notebook. Summary:

1. **Cell 12 (Training baseline ResNet):**
 - `outputs = resnet(inputs, 0, False)` - forward pass with early exit disabled
 - `loss = criterion(outputs, labels)` - standard cross entropy loss
2. **Cell 14 (Evaluating baseline ResNet):**
 - `outputs = resnet(inputs, 0, False)` - same forward pass as training
3. **Cell 16 (Bug fix):**
 - Fixed `resnet_early = resnet.to(device) → resnet_early = resnet_early.to(device)`
 - Fixed optimizer to use `resnet_early.parameters()` instead of `resnet.parameters()`
4. **Cell 18 (Training early exit ResNet):**
 - `outputs = resnet_early.early_train(inputs)` - gets all 4 outputs (3 early exits + final)
 - `loss = 1.0 * criterion(outputs[3], labels) + 0.3 * (criterion(outputs[0], labels) + criterion(outputs[1], labels) + criterion(outputs[2], labels))` - weighted loss with $w=1.0$ for final output and $w=0.3$ for early exits
5. **Cell 20 (Evaluating early exit ResNet):**
 - `outputs, num, curr_entropy = resnet_early(inputs, 0.05, True)` - forward pass with entropy tolerance 0.05 and early exit enabled
 - Fixed bug: changed `total_correct` to `total_early_correct` in the print statement

All TODOs are complete and the code should run correctly.

Implementing a transformer for document summarization

Exported on 12/7/2025 at 18:46:55 PST from Cursor (2.1.46)

User

Now, you will implement a Transformer using fundamental building blocks in PyTorch. You'll apply the Transformer encoder-decoder model to a sequence-to-sequence

NLP task: document summarization. Refer to the Attention is All You Need paper for details on the model

architecture. Here is the notebook that you need to implement the model

[in.@summarize/q_summarize_part_1.ipynb](#)

Cursor

Let me continue reading the notebook to see the remaining parts:

Now I have a complete picture of what needs to be implemented. Let me create a task list and implement the Transformer model step by step.

Let me implement each component. I'll start with the `scaled_dot_product_attention` function:

```
def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False):
    """
    # TODO: implement this function
    # q: [n, tgt_len, d_head]
    # k: [n, src_len, d_head]
    # v: [n, src_len, d_head]
    # key_padding_mask: [n, src_len] - byte tensor with 1 at pad position
    # causal: bool - if True, apply causal masking

    n, tgt_len, d_head = q.shape
    src_len = k.shape[1]

    # Compute attention scores: s_{i,j} = q_i^T k_j / sqrt(d_head)
```

```
# Using einsum: [n, tgt_len, d_head] x [n, src_len, d_head] -> [n, tgt_len, d_head]
scores = torch.einsum('ntd,nsd->nts', q, k) / (d_head ** 0.5)
```

```
# Apply key padding mask: set pad positions to -inf
```

```
if key_padding_mask is not None:
```

```
    # key_padding_mask: [n, src_len] -> expand to [n, 1, src_len] for
    mask = key_padding_mask.unsqueeze(1).bool() # [n, 1, src_len]
    scores = scores.masked_fill(mask, float('-inf'))
```

```
# Apply causal mask: position i can only attend to positions j <= i
```

```
if causal:
```

```
    # Use future_mask which has -inf in upper triangular positions
    causal_mask = future_mask[:tgt_len, :src_len].to(scores.device)
    scores = scores + causal_mask
```

```
# Apply softmax to get attention weights (convert to float32 for nume
```

```
attn_weights = F.softmax(scores.float(), dim=-1).type_as(scores)
```

```
# Apply attention weights to values
```

```
# [n, tgt_len, src_len] x [n, src_len, d_head] -> [n, tgt_len, d_head]
```

```
output = torch.einsum('nts,nsd->ntd', attn_weights, v)
```

```
return output
```

```
#####
```

```
def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False,
```

```
#####
```

```
# TODO: implement this function
```

```
#####
```

```
#####
```

```
->
```

```
def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False,
```

```
#####
```

```
# TODO: implement this function
```

```
#####
```

```
# q: [n, tgt_len, d_head]
```

```
# k: [n, src_len, d_head]
```

```
# v: [n, src_len, d_head]
```

```
# key_padding_mask: [n, src_len] - byte tensor with 1 at pad positions
```

```
# causal: bool - if True, apply causal masking
```

```
n, tgt_len, d_head = q.shape
```

```

src_len = k.shape[1]

# Compute attention scores:  $s_{i,j} = q_i^T k_j / \sqrt{d_{\text{head}}}$ 
# Using einsum: [n, tgt_len, d_head] x [n, src_len, d_head] -> [n, tgt_len, src_len]
scores = torch.einsum('ntd,nsd->nts', q, k) / (d_head ** 0.5)

# Apply key padding mask: set pad positions to -inf
if key_padding_mask is not None:
    # key_padding_mask: [n, src_len] -> expand to [n, 1, src_len] for
    mask = key_padding_mask.unsqueeze(1).bool() # [n, 1, src_len]
    scores = scores.masked_fill(mask, float('-inf'))

# Apply causal mask: position i can only attend to positions j <= i
if causal:
    # Use future_mask which has -inf in upper triangular positions
    causal_mask = future_mask[:tgt_len, :src_len].to(scores.device)
    scores = scores + causal_mask

# Apply softmax to get attention weights (convert to float32 for numerical stability)
attn_weights = F.softmax(scores.float(), dim=-1).type_as(scores)

# Apply attention weights to values
# [n, tgt_len, src_len] x [n, src_len, d_head] -> [n, tgt_len, d_head]
output = torch.einsum('nts,nsd->ntd', attn_weights, v)

```

```

return output

```

```

#####

```

```

def forward(self, q, k, v, key_padding_mask=None, causal=False):
    #####
    # TODO: implement this method
    #####
    #####

```

->

```

def forward(self, q, k, v, key_padding_mask=None, causal=False):
    #####
    # TODO: implement this method
    #####
    # q: [bsz, tgt_len, d_model]
    # k: [bsz, src_len, d_model]
    # v: [bsz, src_len, d_model]
    # key_padding_mask: [bsz, src_len]

```

```

bsz, tgt_len, d_model = q.shape
src_len = k.shape[1]

# Project q, k, v
q = self.q_proj(q) # [bsz, tgt_len, d_model]
k = self.k_proj(k) # [bsz, src_len, d_model]
v = self.v_proj(v) # [bsz, src_len, d_model]

# Reshape to [bsz, seq_len, n_heads, d_head] then rearrange to [k
# Using einops for clarity
q = einops.rearrange(q, 'b t (h d) -> (b h) t d', h=self.n_heads,
k = einops.rearrange(k, 'b s (h d) -> (b h) s d', h=self.n_heads,
v = einops.rearrange(v, 'b s (h d) -> (b h) s d', h=self.n_heads,

# Expand key_padding_mask for n_heads
if key_padding_mask is not None:
    # [bsz, src_len] -> [bsz * n_heads, src_len]
    key_padding_mask = einops.repeat(key_padding_mask, 'b s -> (b
s h) s', h=self.n_heads)

# Apply scaled dot-product attention
attn_output = scaled_dot_product_attention(q, k, v, key_padding_m
# attn_output: [bsz * n_heads, tgt_len, d_head]

# Reshape back to [bsz, tgt_len, d_model]
attn_output = einops.rearrange(attn_output, '(b h) t d -> b t (h
d)', h=self.n_heads)

# Final projection
output = self.o_proj(attn_output) # [bsz, tgt_len, d_model]

return output
#####

```

```

def forward(self, x, padding_mask, encoder_out=None, encoder_padding_
#####
# TODO: implement this method
#####
#####

```

->

```

def forward(self, x, padding_mask, encoder_out=None, encoder_padding_
#####
# TODO: implement this method

```

```
#####
# For encoder: x: [bsz, src_len, d_model], padding_mask: [bsz, sr
# For decoder: x: [bsz, tgt_len, d_model], padding_mask: [bsz, tq
#
#         encoder_out: [bsz, src_len, d_model], encoder_padc

# Self-attention (with causal masking for decoder)
residual = x
if self.is_decoder:
    # Decoder self-attention with causal mask
    x = self.self_attn(x, x, x, key_padding_mask=padding_mask, ca
else:
    # Encoder self-attention (no causal mask)
    x = self.self_attn(x, x, x, key_padding_mask=padding_mask, ca
x = self.self_attn_drop(x)
x = self.self_attn_ln(residual + x)

# Cross-attention (only for decoder)
if self.is_decoder:
    residual = x
    x = self.cross_attn(x, encoder_out, encoder_out, key_padding_
    x = self.cross_attn_drop(x)
    x = self.cross_attn_ln(residual + x)

# Feed-forward network
residual = x
x = self.fc2(F.relu(self.fc1(x)))
x = self.ffn_drop(x)
x = self.ffn_ln(residual + x)

return x
#####
```

```
def make_positions(self, input_ids, padding_mask):
    #####
    # TODO: implement this method
    #####
    #####
```

->

```
def make_positions(self, input_ids, padding_mask):
    #####
    # TODO: implement this method
    #####
```



```

# input_ids: [bsz, seq_len]
# padding_mask: [bsz, seq_len] (not used here, but provided for r
# Returns: [bsz, seq_len] tensor of position indices

bsz, seq_len = input_ids.shape
# Create position indices [0, 1, 2, ..., seq_len-1] and expand to
positions = torch.arange(seq_len, dtype=input_ids.dtype, device=i
positions = positions.unsqueeze(0).expand(bsz, -1)
return positions
#####

```

I've implemented all the required components for the Transformer model. Here's a summary of what was implemented:

1. Scaled Dot-Product Attention

(scaled_dot_product_attention)

This function computes attention as described in the "Attention is All You Need" paper:

- Computes attention scores: $(s_{i,j} = \mathbf{q}_i^T \mathbf{k}_j / \sqrt{d_{\text{head}}})$
- Applies **key padding mask** by setting pad positions to `-inf` before softmax
- Applies **causal mask** using the provided `future_mask` for autoregressive generation
- Applies softmax (in float32 for numerical stability) and computes weighted sum of values

2. Multi-Head Attention

(MultiheadAttention.forward)

- Projects Q, K, V using linear layers (`q_proj`, `k_proj`, `v_proj`)
- Uses `einops.rearrange` to split into multiple attention heads: `[bsz, seq_len, d_model] → [bsz * n_heads, seq_len, d_head]`
- Expands the key padding mask accordingly
- Calls `scaled_dot_product_attention`
- Reshapes output and applies final projection (`o_proj`)

3. Transformer Layer

(TransformerLayer.forward)

Implements encoder/decoder layers following the paper's architecture:

- **Encoder:** Self-attention → LayerNorm → FFN → LayerNorm (with residual connections and dropout)
- **Decoder:** Causal self-attention → LayerNorm → Cross-attention → LayerNorm → FFN → LayerNorm
- Uses ReLU activation in the feed-forward network

4. Position Generation

(`Transformer.make_positions`)

Generates position indices `[0, 1, 2, ..., seq_len-1]` for each sequence in the batch to feed into the learned positional embedding layer.

To run and test your implementation, execute the notebook cells in order. The test cells will verify:

- `test_scaled_dot_product_attention()` - Tests the attention function with various masks
- `test_multihead_attention()` - Compares against PyTorch's `nn.MultiheadAttention`
- `test_transformer_layer()` - Compares against PyTorch's `nn.TransformerEncoderLayer` and `nn.TransformerDecoderLayer`

After the tests pass, you can train the model for 2 epochs on the XSum dataset. The expected validation loss should be close to **5.0** after training.